LAB EXPERIMENTS – ITA04 STATISTICS WITH R PROGRAMMING

1. Create numeric, character, and logical vectors and display type and content.
2. Create labeled matrices (5×4, 3×3, 2×2) filled by row/column.
3. Write an R program to create and display a 3D array with specified rows, columns, and tables. 4. Create arrays from vectors with dimension names, print specific elements. 5. Create and manipulate factor variables (e.g., women's dataset heights, random LETTERS sample). 6. Create an R list containing vectors, matrices, and functions; display contents. 7. Write R programs for basic tasks: Factors of a number, generate a vector of 10 random integers between -50 and 50, print numbers 1–100 with FizzBuzz logic.
8. Generate random numbers from a normal distribution; count occurrences.
9. Create empty plots with specified axis limits.
10. Create and explore a data frame exam_data with name, score, attempts, and qualify fields. Perform extract, add row/column, sort, save to file.
11. Write an R program to read a .csv file and display contents.
12. Perform data reshaping on airquality dataset: melt, cast, compute monthly averages for Ozone, Solar.R, Wind, and Temperature.
13. Combine multiple arrays row-wise.
14. Explore and manipulate ChickWeight dataset (sorting, melting, casting by Diet). 15. Perform EDA on iris dataset: dimensions, summary, standard deviation, quantiles, grouping by Species, pivot table, categorical grouping with Sepal.Length categories.
16. Explore USArrests dataset: summary statistics, state with largest rape arrests, max & min murder rates, correlation among features, states above median assault arrests and bottom 25% for murder, visualization with histogram, density, scatterplots, bar graphs.
17. Explore Titanic dataset: bar chart of survival vs class, modify plot by gender, histogram of Age.
18. Create graphs in R: boxplot, histogram, bar plot, line chart, scatter plot.
19. Build a regression model on advertising dataset (Sales ~ Spend) and predict Sales. 20. Create multiple regression model using ChickWeight dataset with "Time" and "Diet" as predictors; predict weight and compute model error.
21. Randomly split iris dataset into train/test (80/20), build logistic regression (Species ~ Petal.Length + Petal.Width), predict, and evaluate with confusion matrix.

Experiment 1: Vectors

Question

1. Create numeric, character, and logical vectors and display type and content.

Code

```
num_vec <- c(10, 20.5, 30, 40)
chr_vec <- c("R", "Python", "Java")
log_vec <- c(TRUE, FALSE, TRUE)


cat("Numeric vector   :", num_vec, "\n")
cat("Type:", typeof(num_vec), "| Class:", class(num_vec), "\n\n")


cat("Character vector :", chr_vec, "\n")
cat("Type:", typeof(chr_vec), "| Class:", class(chr_vec), "\n\n")


cat("Logical vector   :", log_vec, "\n")
cat("Type:", typeof(log_vec), "| Class:", class(log_vec), "\n")
```

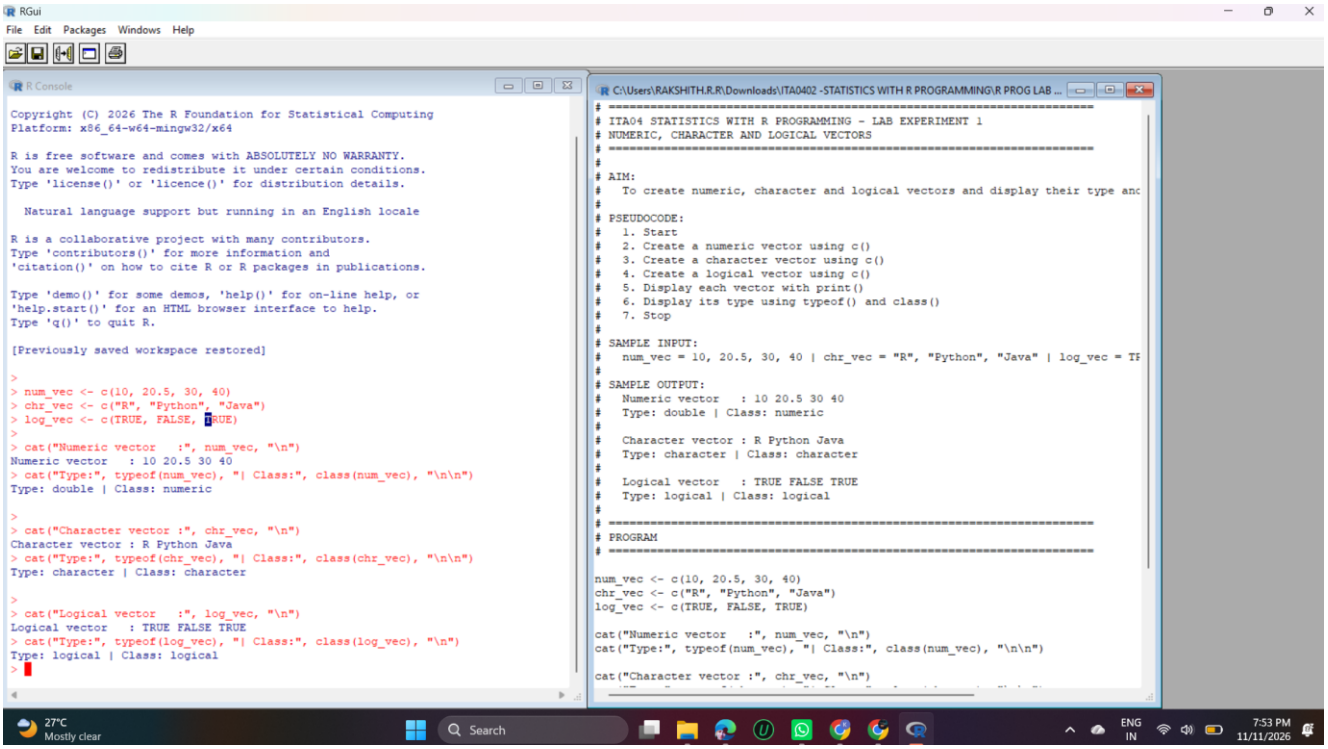
Output

```
Numeric vector   : 10 20.5 30 40
Type: double | Class: numeric


Character vector : R Python Java
Type: character | Class: character


Logical vector   : TRUE FALSE TRUE
Type: logical | Class: logical
```

Output Screenshot



Output – Experiment 1

Explanation

A vector is the basic data structure in R. The c() function combines values into a vector.

num_vec holds numbers, chr_vec holds text in quotes and log_vec holds TRUE/FALSE values.

typeof() shows how R stores the data internally (20.5 makes the numeric vector type “double”), and class() shows the object class (numeric, character, logical).

cat() prints the contents, so the output confirms the content and type of all three vectors.

Experiment 2: Labeled Matrices

Question

2. Create labeled matrices (5×4, 3×3, 2×2) filled by row/column.

Code

```
# 5 x 4 matrix filled BY ROW
m1 <- matrix(1:20, nrow = 5, ncol = 4, byrow = TRUE,
             dimnames = list(c("R1","R2","R3","R4","R5"),
                             c("C1","C2","C3","C4")))
print("5 x 4 matrix (filled by row):")
print(m1)


# 3 x 3 matrix filled BY COLUMN
m2 <- matrix(1:9, nrow = 3, ncol = 3, byrow = FALSE,
             dimnames = list(c("a","b","c"), c("x","y","z")))
print("3 x 3 matrix (filled by column):")
print(m2)

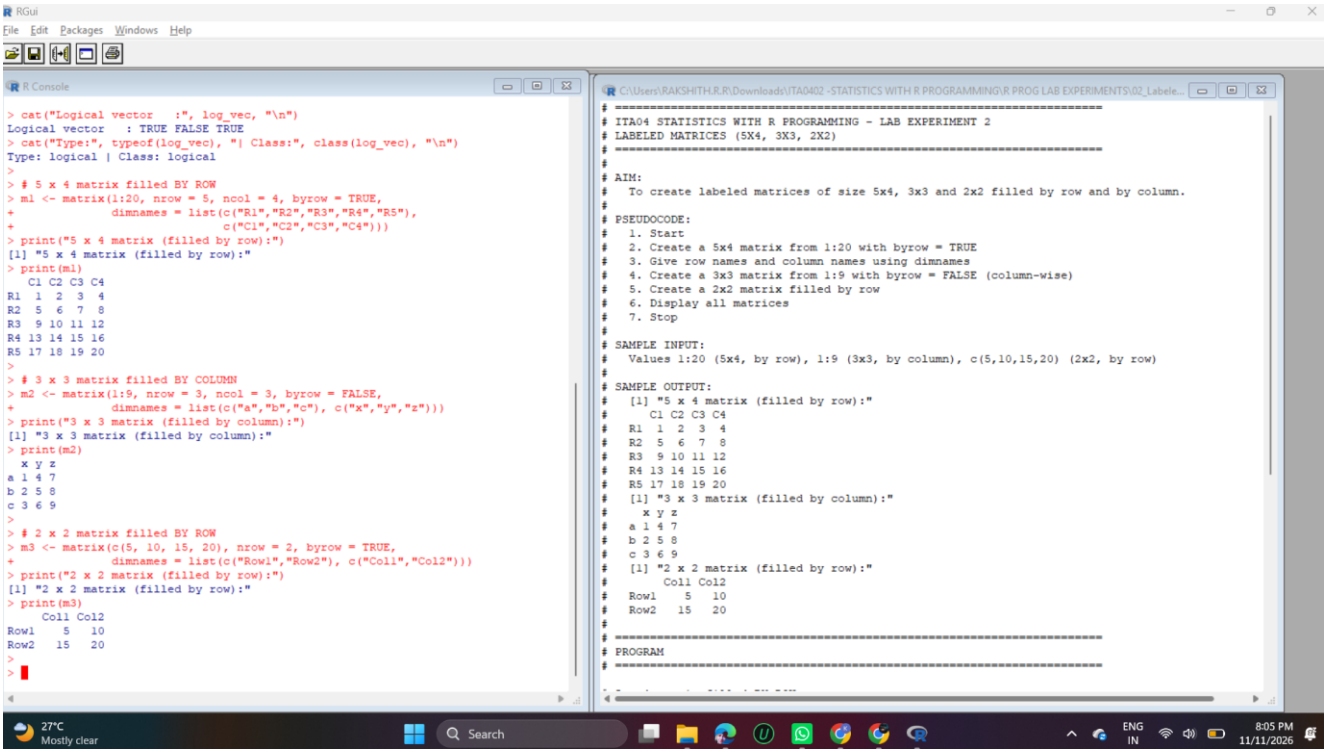

# 2 x 2 matrix filled BY ROW
m3 <- matrix(c(5, 10, 15, 20), nrow = 2, byrow = TRUE,
             dimnames = list(c("Row1","Row2"), c("Col1","Col2")))
print("2 x 2 matrix (filled by row):")
print(m3)
```

Output

```
[1] "5 x 4 matrix (filled by row):"
   C1 C2 C3 C4
R1  1  2  3  4
R2  5  6  7  8
R3  9 10 11 12
R4 13 14 15 16
R5 17 18 19 20
[1] "3 x 3 matrix (filled by column):"
  x y z
a 1 4 7
b 2 5 8
c 3 6 9
```

```
[1] "2 x 2 matrix (filled by row):"
     Col1 Col2
Row1    5   10
Row2   15   20
```

Output Screenshot



Output – Experiment 2

Explanation

matrix(data, nrow, ncol, byrow) builds a matrix. With byrow = TRUE the values are filled row by row (the 5×4 and 2×2 matrices); with byrow = FALSE, the default, they are filled column by column (the 3×3 matrix).

dimnames = list(row_names, column_names) labels the rows and columns, so the matrices print with names like R1–R5 and C1–C4 instead of [1,] and [,1].

The output shows 1–20 across the rows of the 5×4 matrix, 1–9 down the columns of the 3×3 matrix and 5, 10, 15, 20 across the rows of the 2×2 matrix.

Experiment 3: 3D Array

Question

3. Write an R program to create and display a 3D array with specified rows, columns, and tables.

Code

```
arr <- array(1:24, dim = c(4, 3, 2))   # 4 rows, 3 columns, 2 tables
print(arr)
cat("Dimensions of the array:", dim(arr), "\n")
```

Output

```
, , 1

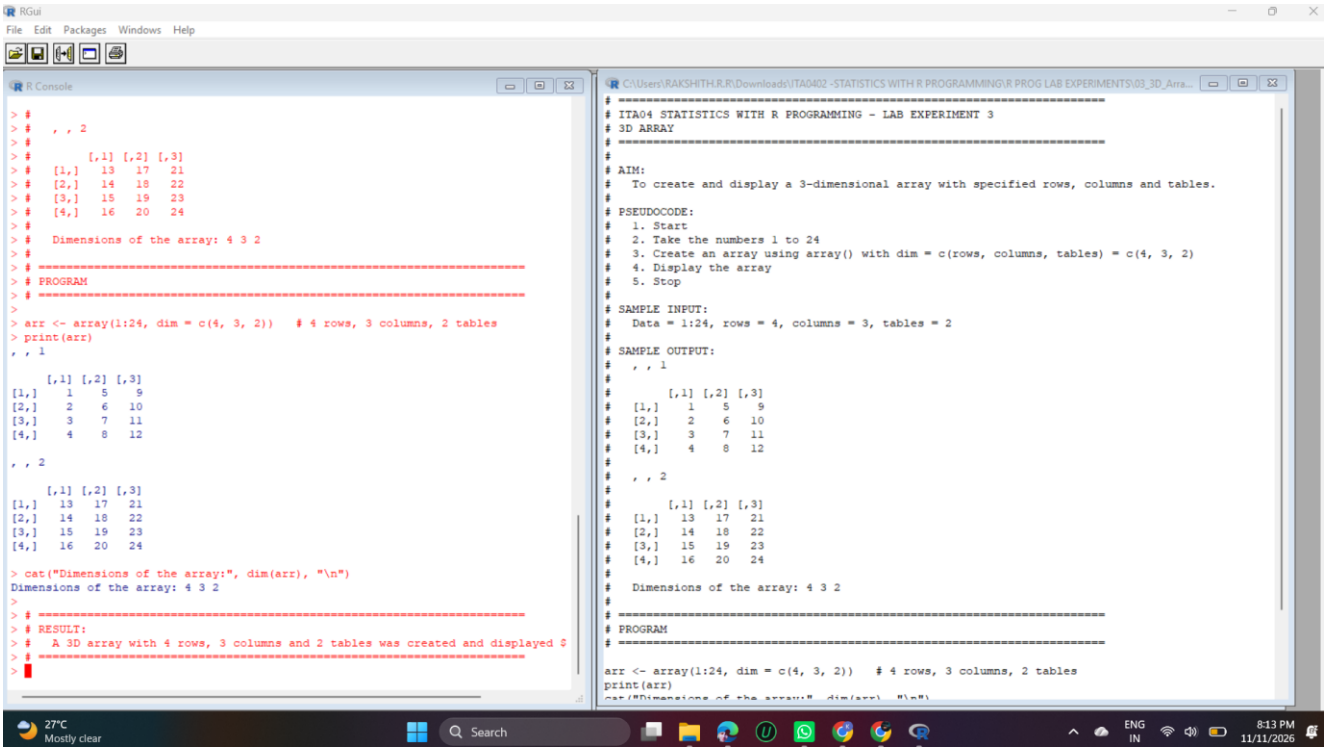
     [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12

, , 2

     [,1] [,2] [,3]
[1,]   13   17   21
[2,]   14   18   22
[3,]   15   19   23
[4,]   16   20   24

Dimensions of the array: 4 3 2
```

Output Screenshot



Output – Experiment 3

Explanation

array(data, dim = c(rows, columns, tables)) creates a multi-dimensional array. Here dim = c(4, 3, 2) gives 4 rows, 3 columns and 2 tables, so 4 × 3 × 2 = 24 elements are needed, which is exactly 1:24.

R fills each table column by column, so the first table holds 1–12 and the second holds 13–24. They are printed as “, , 1” and “, , 2”.

dim() returns the dimensions 4 3 2, confirming the structure.

Experiment 4: Array with Dimension Names

Question

4. Create arrays from vectors with dimension names, print specific elements.

Code

```
vec1 <- c(5, 9, 3)
vec2 <- c(10, 11, 12, 13, 14, 15)

result <- array(c(vec1, vec2), dim = c(3, 3, 2),
                dimnames = list(c("Row1","Row2","Row3"),
                                c("Col1","Col2","Col3"),
                                c("Matrix1","Matrix2")))
print(result)

cat("\nElement at Row3, Col2 of Matrix1 :", result[3, 2, 1], "\n")
cat("Element at Row1, Col3 of Matrix2 :", result["Row1", "Col3", "Matrix2"], "\n")
print("Second matrix:")
print(result[, , 2])
```

Output

```
, , Matrix1


     Col1 Col2 Col3
Row1    5   10   13
Row2    9   11   14
Row3    3   12   15


, , Matrix2

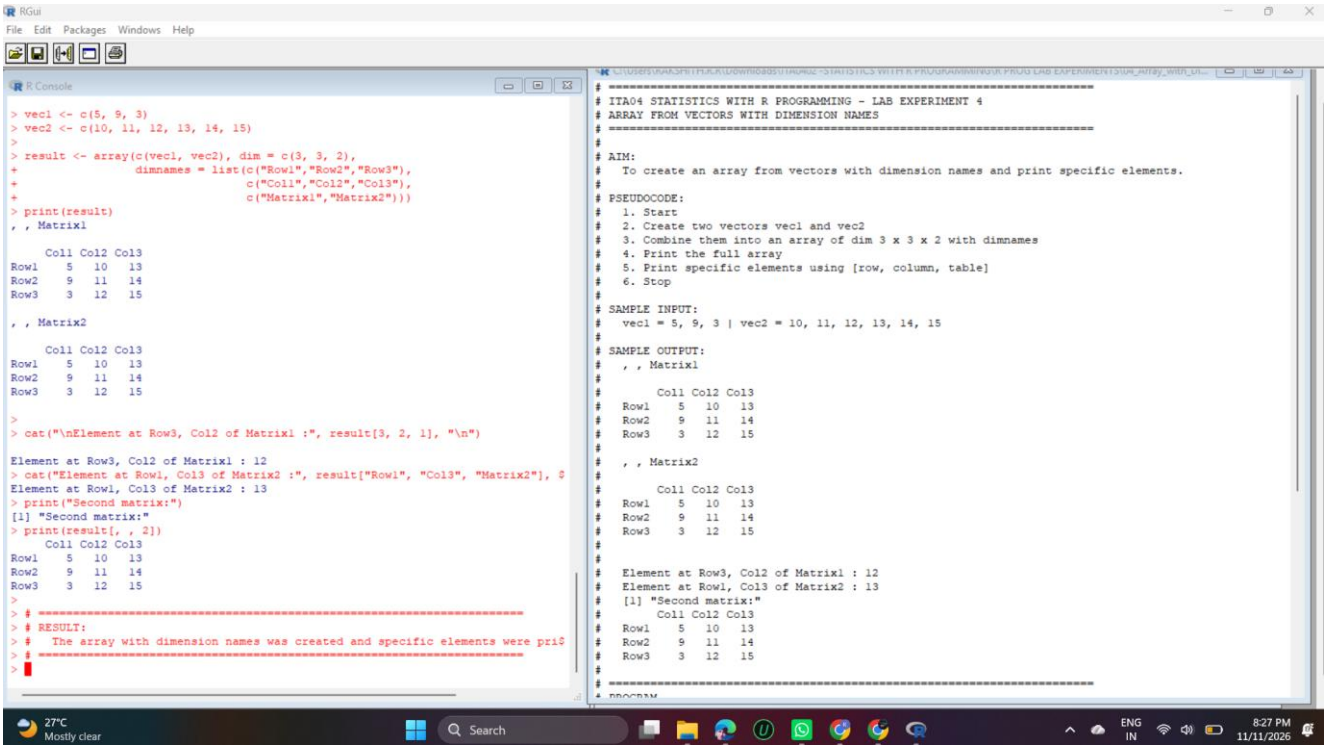

     Col1 Col2 Col3
Row1    5   10   13
Row2    9   11   14
Row3    3   12   15




Element at Row3, Col2 of Matrix1 : 12
Element at Row1, Col3 of Matrix2 : 13
[1] "Second matrix:"
```

```
     Col1 Col2 Col3
Row1    5   10   13
Row2    9   11   14
Row3    3   12   15
```

Output Screenshot



Output – Experiment 4

Explanation

c(vec1, vec2) joins the two vectors into 9 values, but a 3×3×2 array needs 18. R recycles (repeats) the 9 values, which is why Matrix2 is identical to Matrix1. This is normal R behaviour.

dimnames takes a list of three name vectors, for the rows, the columns and the matrices, so every element can be addressed by name.

Elements are accessed as [row, column, matrix]. result[3, 2, 1] gives 12, result["Row1", "Col3", "Matrix2"] gives 13 using names, and result[, , 2] prints the whole second matrix.

Experiment 5: Factor Variables

Question

5. Create and manipulate factor variables (e.g., women's dataset heights, random LETTERS sample).

Code

```
# Part 1: women dataset
h <- factor(women$height)
print(levels(h))
cat("Number of levels:", nlevels(h), "\n")
print(summary(h)[1:5])

# Part 2: random sample of LETTERS
set.seed(1)
s <- sample(LETTERS, 10, replace = TRUE)
f <- factor(s)
print(s)
print(f)
print(table(f))

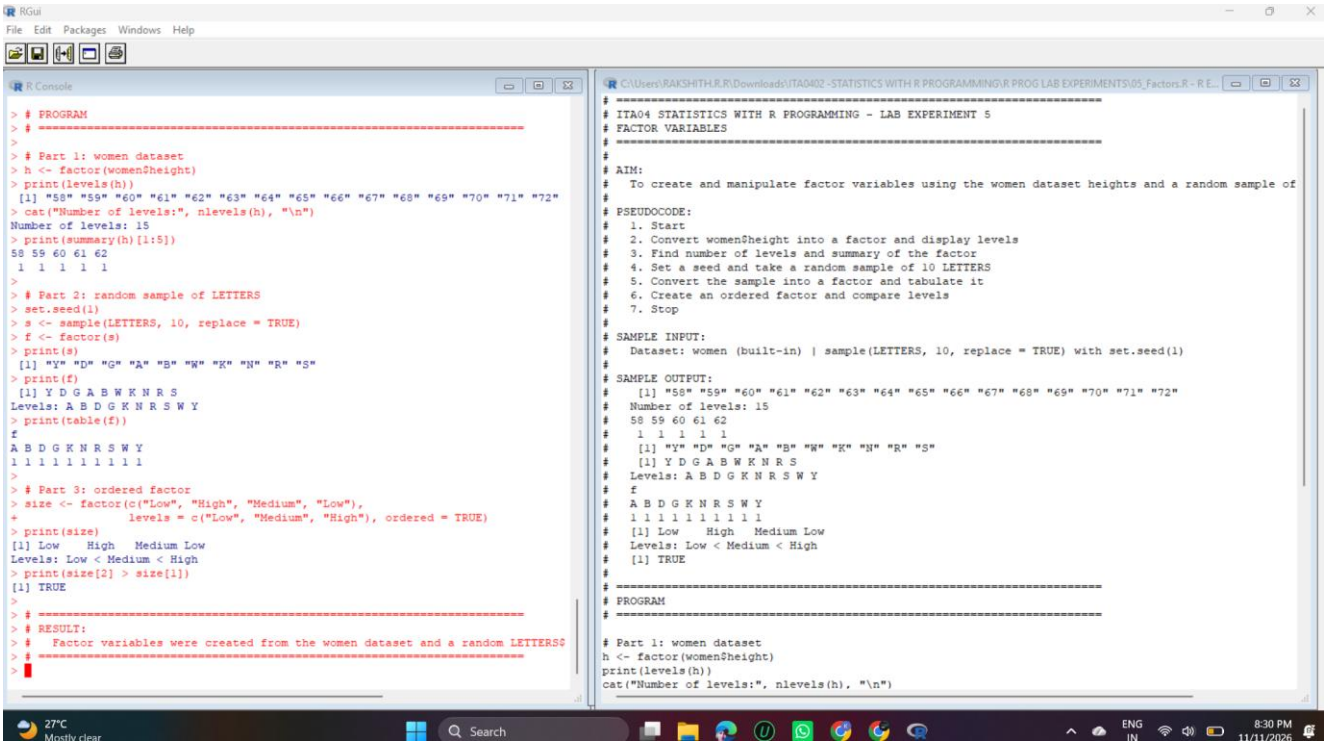
# Part 3: ordered factor
size <- factor(c("Low", "High", "Medium", "Low"),
               levels = c("Low", "Medium", "High"), ordered = TRUE)
print(size)
print(size[2] > size[1])
```

Output

```
 [1] "58" "59" "60" "61" "62" "63" "64" "65" "66" "67" "68" "69" "70" "71" "72"
Number of levels: 15
58 59 60 61 62
 1  1  1  1  1
 [1] "Y" "D" "G" "A" "B" "W" "K" "N" "R" "S"
 [1] Y D G A B W K N R S
Levels: A B D G K N R S W Y
f
A B D G K N R S W Y
1 1 1 1 1 1 1 1 1 1
[1] Low    High   Medium Low
```

> Levels: Low < Medium < High
>
> [1] TRUE

Output Screenshot



Output – Experiment 5

Explanation

factor() stores categorical data as levels. women$height has 15 different heights (58 to 72), so the factor has 15 levels and each level occurs once.

set.seed(1) makes the random result repeatable. sample(LETTERS, 10, replace = TRUE) picks 10 random capital letters, and factor() turns them into a factor whose levels are sorted alphabetically. table() counts how often each level occurs.

The ordered factor uses levels Low < Medium < High. Because the order is defined, comparisons are allowed: size[2] (High) > size[1] (Low) returns TRUE.

Experiment 6: List

Question

6. Create an R list containing vectors, matrices, and functions; display contents.

Code

```
my_vec <- c(1, 2, 3, 4, 5)
my_mat <- matrix(1:4, nrow = 2)
square <- function(x) {
  return(x^2)
}

my_list <- list(vector = my_vec, matrix = my_mat, func = square)

print(my_list)

cat("\nVector element :", my_list$vector, "\n")
cat("Function call square(4) =", my_list$func(4), "\n")
```

Output

```
$vector
[1] 1 2 3 4 5

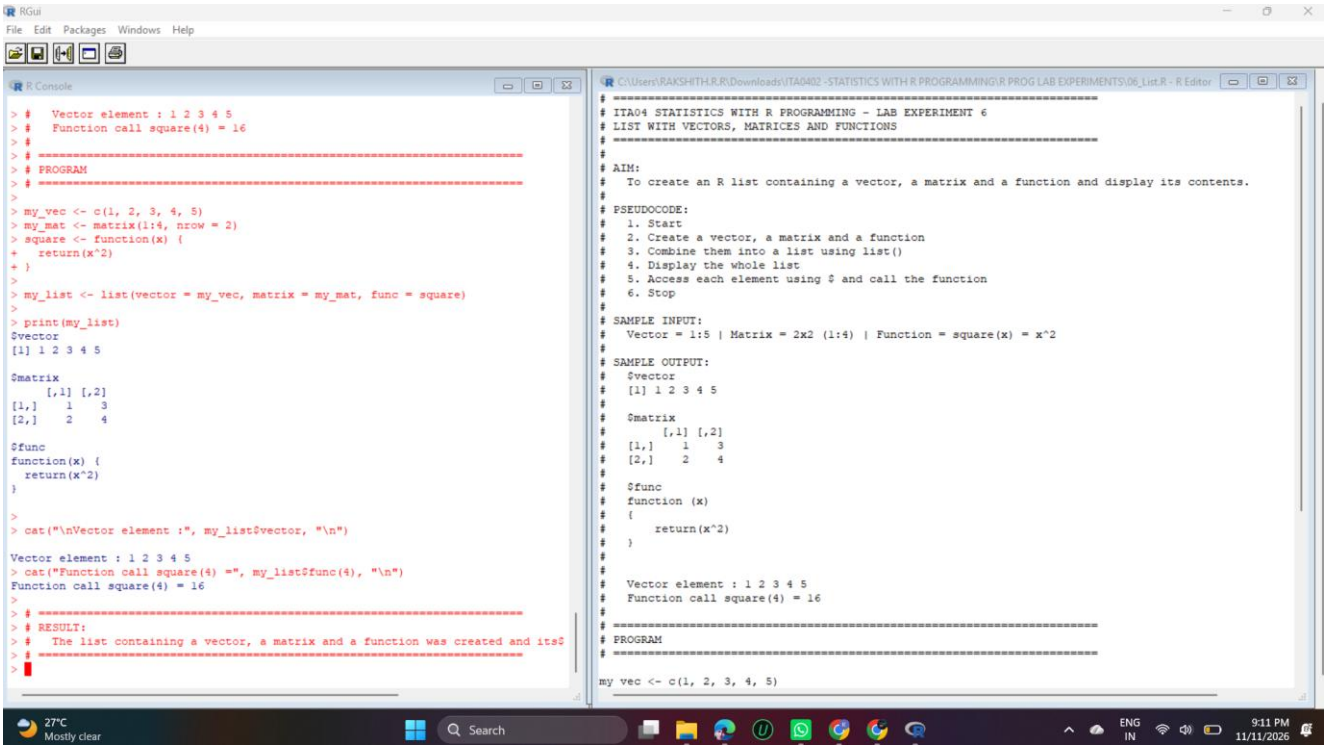
$matrix
     [,1] [,2]
[1,]    1    3
[2,]    2    4

$func
function(x) {
  return(x^2)
}



Vector element : 1 2 3 4 5
Function call square(4) = 16
```

Output Screenshot



Output – Experiment 6

Explanation

A list can store elements of different types in one object. Here it holds a numeric vector, a 2×2 matrix and a function.

Elements are accessed by name with $. my_list$vector gives the vector and my_list$func(4) calls the stored function, returning $4^2 = 16$.

print(my_list) displays all three components, including the source code of the function.

Experiment 7: Factors, Random Integers and FizzBuzz

Question

7. Write R programs for basic tasks: Factors of a number, generate a vector of 10 random integers between -50 and 50, print numbers 1–100 with FizzBuzz logic.

Code

```
# (a) Factors of a number
n <- 24
cat("Factors of", n, "are: ")
for (i in 1:n) {
  if (n %% i == 0) cat(i, "")
}
cat("\n\n")


# (b) 10 random integers between -50 and 50
set.seed(10)
rand_vec <- sample(-50:50, 10, replace = TRUE)
cat("Random integers:", rand_vec, "\n\n")


# (c) FizzBuzz 1 to 100
for (i in 1:100) {
  if (i %% 15 == 0) {
    print("FizzBuzz")
  } else if (i %% 3 == 0) {
    print("Fizz")
  } else if (i %% 5 == 0) {
    print("Buzz")
  } else {
    print(i)
  }
}
```

Output

```
Factors of 24 are: 1 2 3 4 6 8 12 24

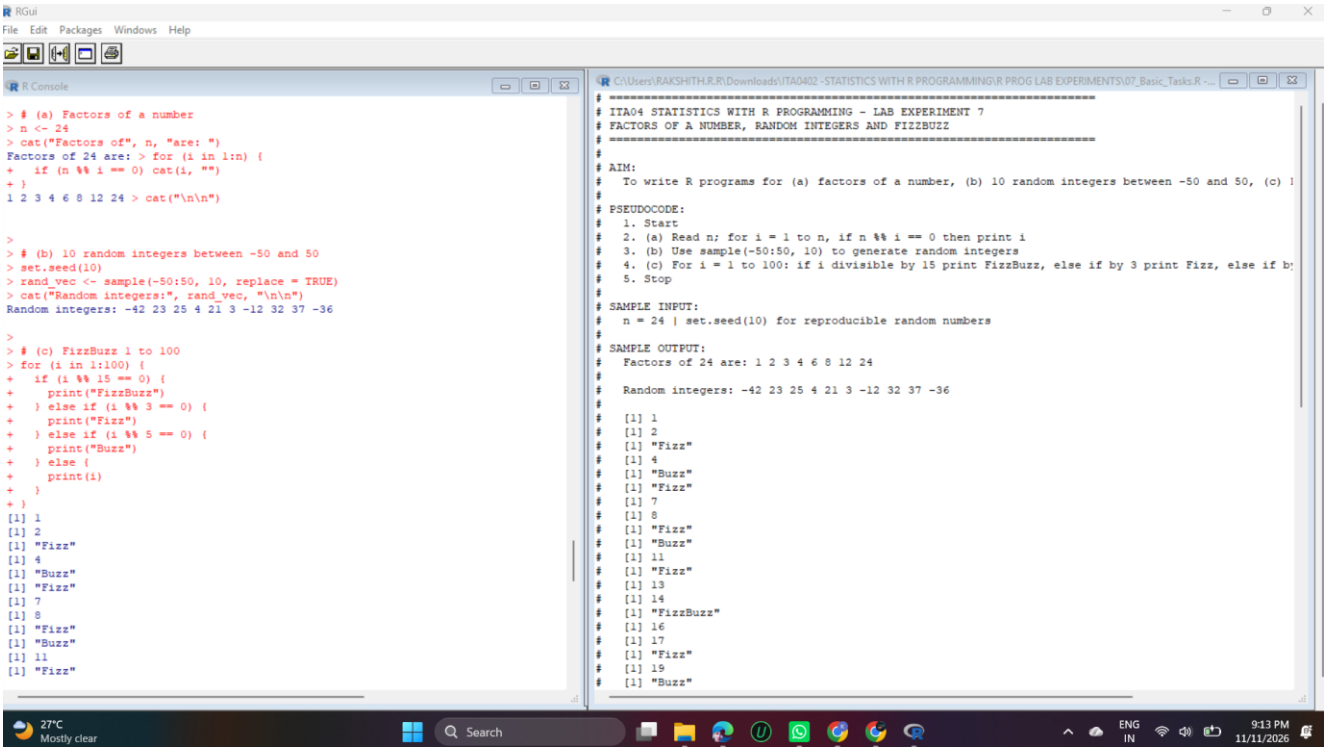

Random integers: -42 23 25 4 21 3 -12 32 37 -36
```

```
[1] 1
[1] 2
[1] "Fizz"
[1] 4
[1] "Buzz"
[1] "Fizz"
[1] 7
[1] 8
[1] "Fizz"
[1] "Buzz"
[1] 11
[1] "Fizz"
[1] 13
[1] 14
[1] "FizzBuzz"
[1] 16
[1] 17
[1] "Fizz"
[1] 19
[1] "Buzz"
[1] "Fizz"
[1] 22
[1] 23
[1] "Fizz"
[1] "Buzz"
[1] 26
[1] "Fizz"
[1] 28
[1] 29
[1] "FizzBuzz"
[1] 31
[1] 32
[1] "Fizz"
[1] 34
[1] "Buzz"
[1] "Fizz"
[1] 37
[1] 38
[1] "Fizz"
```

```
[1] "Buzz"
[1] 41
[1] "Fizz"
[1] 43
[1] 44
[1] "FizzBuzz"
[1] 46
[1] 47
[1] "Fizz"
[1] 49
[1] "Buzz"
[1] "Fizz"
[1] 52
[1] 53
[1] "Fizz"
[1] "Buzz"
[1] 56
[1] "Fizz"
[1] 58
[1] 59
[1] "FizzBuzz"
[1] 61
[1] 62
[1] "Fizz"
[1] 64
[1] "Buzz"
[1] "Fizz"
[1] 67
[1] 68
[1] "Fizz"
[1] "Buzz"
[1] 71
[1] "Fizz"
[1] 73
[1] 74
[1] "FizzBuzz"
[1] 76
[1] 77
[1] "Fizz"
```

```
[1] 79
[1] "Buzz"
[1] "Fizz"
[1] 82
[1] 83
[1] "Fizz"
[1] "Buzz"
[1] 86
[1] "Fizz"
[1] 88
[1] 89
[1] "FizzBuzz"
[1] 91
[1] 92
[1] "Fizz"
[1] 94
[1] "Buzz"
[1] "Fizz"
[1] 97
[1] 98
[1] "Fizz"
[1] "Buzz"
```

Output Screenshot



Output – Experiment 7

Explanation

(a) The for loop checks every number from 1 to n and prints those where n %% i == 0 (remainder zero). The factors of 24 are 1, 2, 3, 4, 6, 8, 12 and 24.

(b) sample(-50:50, 10, replace = TRUE) picks 10 random integers from -50 to 50. set.seed(10) makes the same numbers appear every time.

(c) FizzBuzz prints “FizzBuzz” for multiples of 15, “Fizz” for multiples of 3, “Buzz” for multiples of 5 and the number itself otherwise. The multiple-of-15 test must come first, otherwise those numbers would only print “Fizz”.

Experiment 8: Normal Distribution

Question

8. Generate random numbers from a normal distribution; count occurrences.

Code

```
set.seed(123)
x <- rnorm(20, mean = 5, sd = 2)
print(round(x, 2))


r <- round(x)
cat("\nRounded values:", r, "\n")


cat("\nOccurrences of each value:\n")
print(table(r))


cat("\nNumbers greater than the mean (", round(mean(x), 2), "):", sum(x > mean(x)), "\n")
```

Output

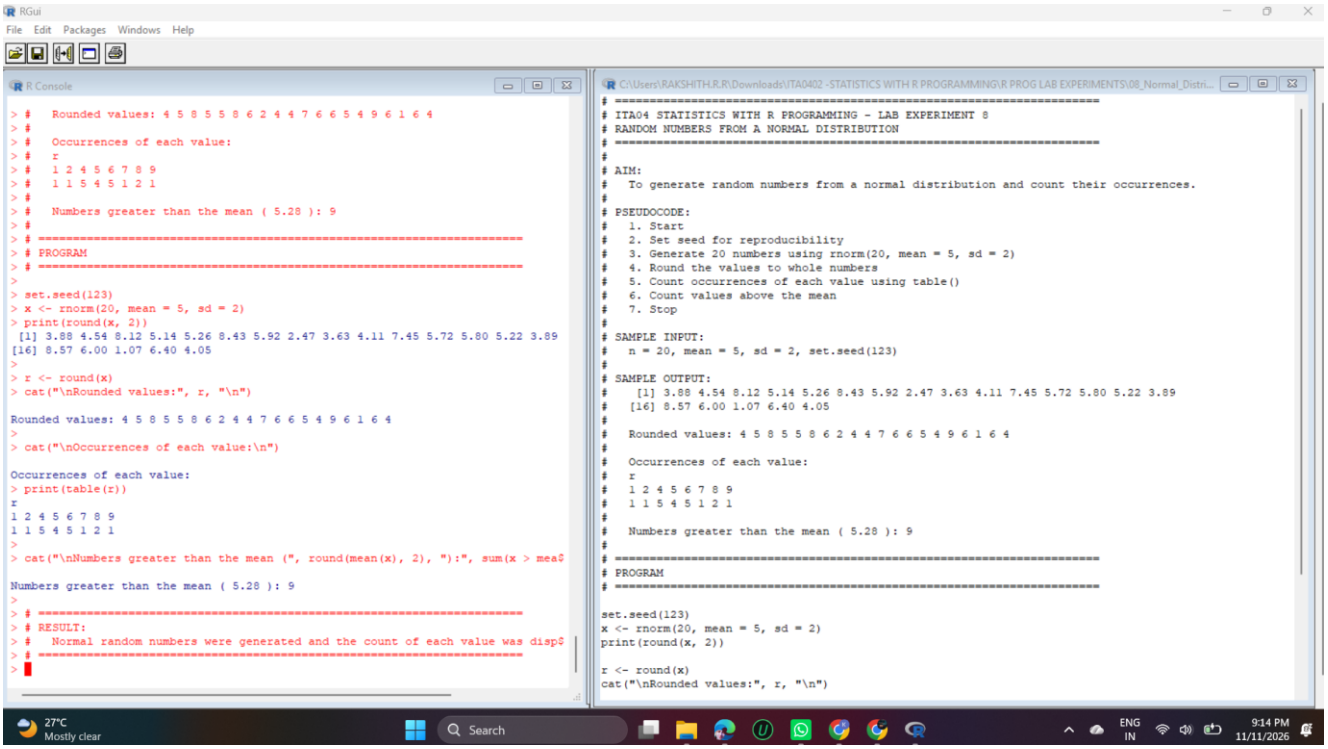
```
 [1] 3.88 4.54 8.12 5.14 5.26 8.43 5.92 2.47 3.63 4.11 7.45 5.72 5.80 5.22 3.89
[16] 8.57 6.00 1.07 6.40 4.05


Rounded values: 4 5 8 5 5 8 6 2 4 4 7 6 6 5 4 9 6 1 6 4


Occurrences of each value:
r
1 2 4 5 6 7 8 9
1 1 5 4 5 1 2 1


Numbers greater than the mean ( 5.28 ): 9
```

Output Screenshot



Output – Experiment 8

Explanation

rnorm(20, mean = 5, sd = 2) generates 20 random values from a normal distribution centred around 5 with a spread of 2. set.seed(123) fixes the random numbers so the output is repeatable.

round() converts the values to whole numbers and table() counts the occurrences of each. The values 4 and 6 occur five times each, 5 occurs four times, and 1, 2, 7 and 9 occur once each.

The mean of the generated data is 5.28, and sum(x > mean(x)) counts the 9 values that are greater than the mean.

Experiment 9: Empty Plots

Question

9. Create empty plots with specified axis limits.

Code

```
par(mfrow = c(1, 2))   # two plots side by side


plot(1, type = "n", xlim = c(0, 10), ylim = c(0, 50),
     xlab = "X axis", ylab = "Y axis", main = "Empty plot 1")

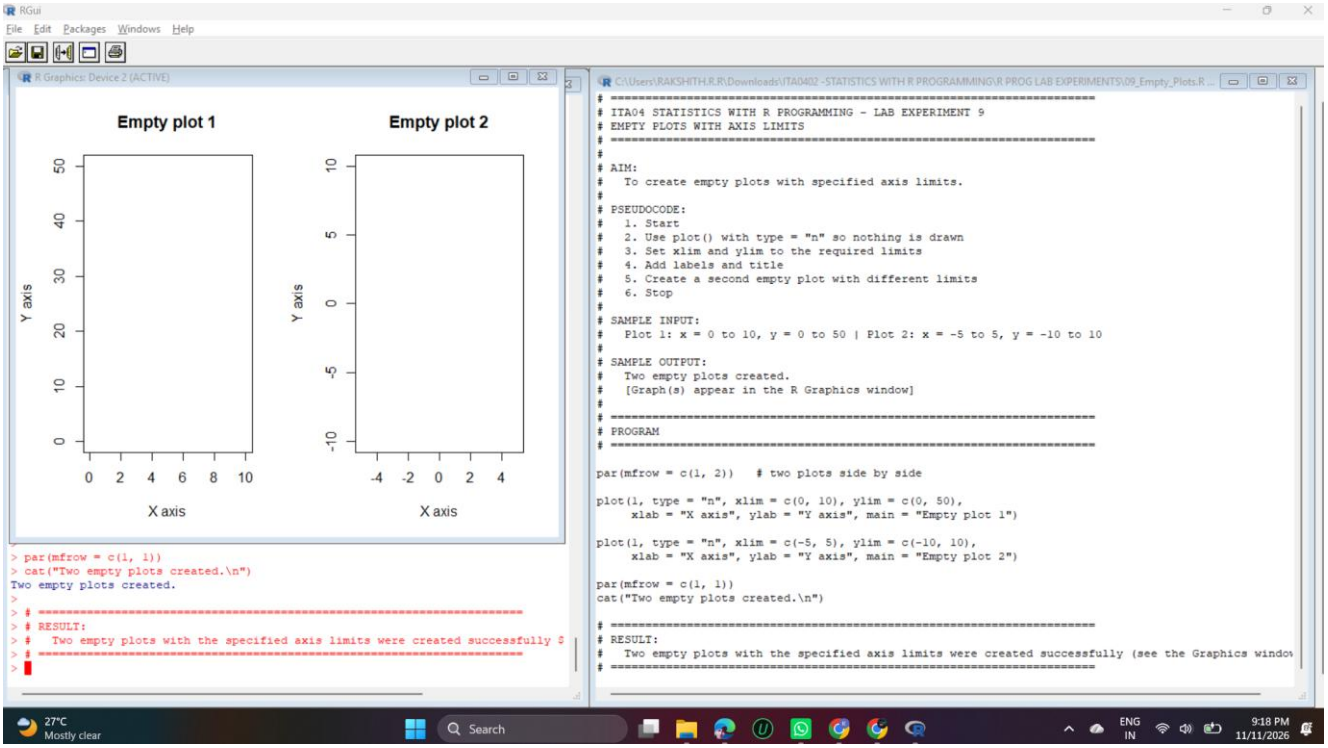

plot(1, type = "n", xlim = c(-5, 5), ylim = c(-10, 10),
     xlab = "X axis", ylab = "Y axis", main = "Empty plot 2")


par(mfrow = c(1, 1))
cat("Two empty plots created.\n")
```

Output

```
Two empty plots created.
```

Output Screenshot



Output – Experiment 9

Explanation

plot(1, type = "n") draws only the axes and the frame without any points or lines, creating an empty plot.

xlim and ylim set the axis ranges (0–10 and 0–50 for plot 1, -5 to 5 and -10 to 10 for plot 2), and xlab, ylab and main add the labels and title.

par(mfrow = c(1, 2)) arranges two plots side by side. Such empty plots are used as a blank canvas to which points or lines are added later.

Experiment 10: Data Frame exam_data

Question

10. Create and explore a data frame exam_data with name, score, attempts, and qualify fields. Perform extract, add row/column, sort, save to file.

Code

```
exam_data <- data.frame(
  name = c("Anastasia", "Dima", "Katherine", "James", "Emily",
           "Michael", "Matthew", "Laura", "Kevin", "Jonas"),
  score = c(12.5, 9, 16.5, NA, 9, 20, 14.5, 13.5, 8, 19),
  attempts = c(1, 3, 2, 3, 2, 3, 1, 1, 2, 1),
  qualify = c("yes", "no", "yes", "no", "no", "yes", "yes", "no", "no", "yes"),
  stringsAsFactors = FALSE
)

print(exam_data)
str(exam_data)

# Extract
cat("\nFirst two rows:\n");          print(exam_data[1:2, ])
cat("\nName and score columns:\n");   print(exam_data[, c("name", "score")])

# Add a new row
new_row <- data.frame(name = "Suresh", score = 15, attempts = 1, qualify = "yes")
exam_data <- rbind(exam_data, new_row)

# Add a new column
exam_data$country <- "India"
cat("\nAfter adding row and column:\n"); print(exam_data)

# Sort by score (descending)
sorted_data <- exam_data[order(-exam_data$score), ]
cat("\nSorted by score:\n"); print(sorted_data)

# Save to file
write.csv(exam_data, "exam_data.csv", row.names = FALSE)
```

```
cat("\nFile exam_data.csv saved in:", getwd(), "\n")
```

Output

```
       name score attempts qualify
1  Anastasia  12.5        1     yes
2       Dima   9.0        3      no
3  Katherine  16.5        2     yes
4      James    NA        3      no
5      Emily   9.0        2      no
6    Michael  20.0        3     yes
7    Matthew  14.5        1     yes
8      Laura  13.5        1      no
9      Kevin   8.0        2      no
10     Jonas  19.0        1     yes
'data.frame':    10 obs. of  4 variables:
 $ name    : chr  "Anastasia" "Dima" "Katherine" "James" ...
 $ score   : num  12.5 9 16.5 NA 9 20 14.5 13.5 8 19
 $ attempts: num  1 3 2 3 2 3 1 1 2 1
 $ qualify : chr  "yes" "no" "yes" "no" ...

First two rows:
       name score attempts qualify
1 Anastasia  12.5        1     yes
2      Dima   9.0        3      no

Name and score columns:
       name score
1  Anastasia  12.5
2       Dima   9.0
3  Katherine  16.5
4      James    NA
5      Emily   9.0
6    Michael  20.0
7    Matthew  14.5
8      Laura  13.5
9      Kevin   8.0
10     Jonas  19.0

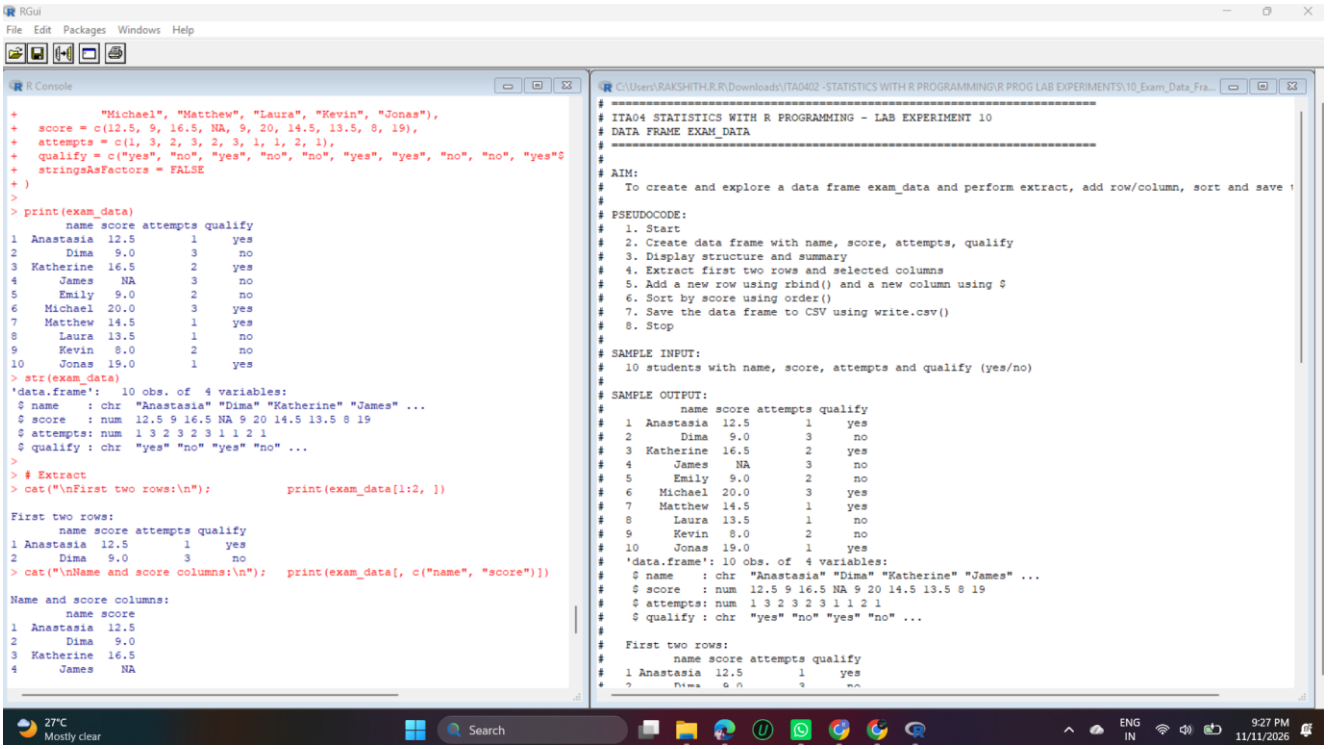
After adding row and column:
```

```
      name score attempts qualify country
1  Anastasia  12.5        1     yes   India
2       Dima   9.0        3      no   India
3  Katherine  16.5        2     yes   India
4      James    NA        3      no   India
5      Emily   9.0        2      no   India
6    Michael  20.0        3     yes   India
7    Matthew  14.5        1     yes   India
8      Laura  13.5        1      no   India
9      Kevin   8.0        2      no   India
10     Jonas  19.0        1     yes   India
11    Suresh  15.0        1     yes   India

Sorted by score:
      name score attempts qualify country
6    Michael  20.0        3     yes   India
10     Jonas  19.0        1     yes   India
3  Katherine  16.5        2     yes   India
11    Suresh  15.0        1     yes   India
7    Matthew  14.5        1     yes   India
8      Laura  13.5        1      no   India
1  Anastasia  12.5        1     yes   India
2       Dima   9.0        3      no   India
5      Emily   9.0        2      no   India
9      Kevin   8.0        2      no   India
4      James    NA        3      no   India

File exam_data.csv saved in: C:/Users/RAKSHITH.R.R/OneDrive/文档
```

Output Screenshot



Output – Experiment 10

Explanation

data.frame() creates a table with the columns name, score, attempts and qualify, and str() shows its structure.

Extract: exam_data[1:2,] selects the first two rows and exam_data[, c("name", "score")] selects two columns.

Add: rbind() appends a new row (Suresh) and exam_data$country <- "India" adds a new column.

Sort: order(-exam_data$score) sorts by score in descending order, and the NA value (James) is placed last.

Save: write.csv() writes the data frame to exam_data.csv in the working directory, and getwd() prints that location.

Experiment 11: Read a CSV File

Question

11. Write an R program to read a .csv file and display contents.

Code

```
# Create a sample CSV file
students <- data.frame(id = 1:5,
                       name = c("Arun", "Bala", "Charan", "Divya", "Esha"),
                       marks = c(85, 72, 90, 66, 78))
write.csv(students, "students.csv", row.names = FALSE)

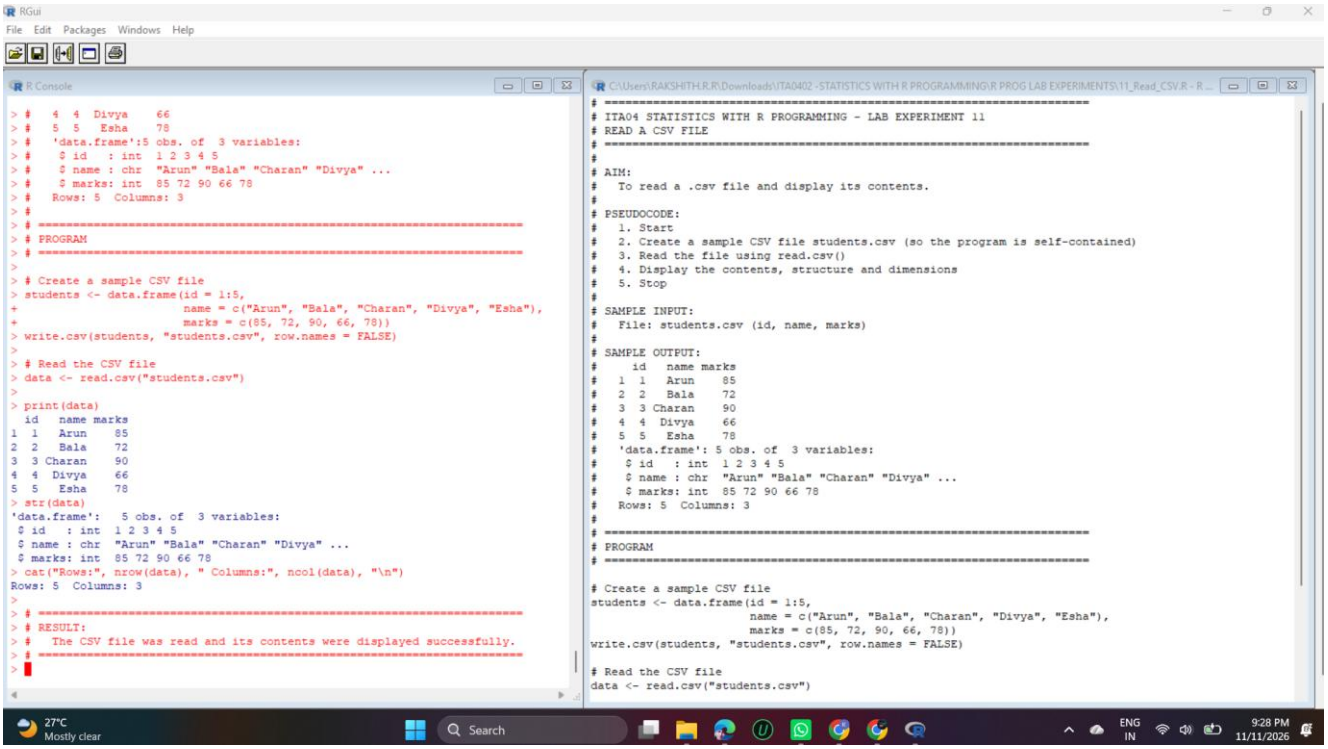

# Read the CSV file
data <- read.csv("students.csv")


print(data)
str(data)
cat("Rows:", nrow(data), " Columns:", ncol(data), "\n")
```

Output

```
  id   name marks
1  1   Arun    85
2  2   Bala    72
3  3 Charan    90
4  4  Divya    66
5  5   Esha    78
'data.frame':    5 obs. of  3 variables:
 $ id   : int  1 2 3 4 5
 $ name : chr  "Arun" "Bala" "Charan" "Divya" ...
 $ marks: int  85 72 90 66 78
Rows: 5  Columns: 3
```

Output Screenshot



Output – Experiment 11

Explanation

write.csv() first creates a sample file, students.csv, so the program is self-contained.

read.csv() reads the file back into a data frame, and print() displays its contents. str() shows that id and marks are integers and name is text. nrow() and ncol() report 5 rows and 3 columns.

Experiment 12: Airquality Reshaping

Question

12. Perform data reshaping on airquality dataset: melt, cast, compute monthly averages for Ozone, Solar.R, Wind, and Temperature.

Code

```
if (!require(reshape2)) install.packages("reshape2")
library(reshape2)


names(airquality) <- tolower(names(airquality))
print(head(airquality))


# MELT: wide to long
aq_melt <- melt(airquality, id.vars = c("month", "day"), na.rm = TRUE)
cat("\nMolten data:\n")
print(head(aq_melt))


# CAST: long to wide with monthly averages
monthly_avg <- dcast(aq_melt, month ~ variable, mean)
cat("\nMonthly averages:\n")
print(round(monthly_avg, 2))
```

Output

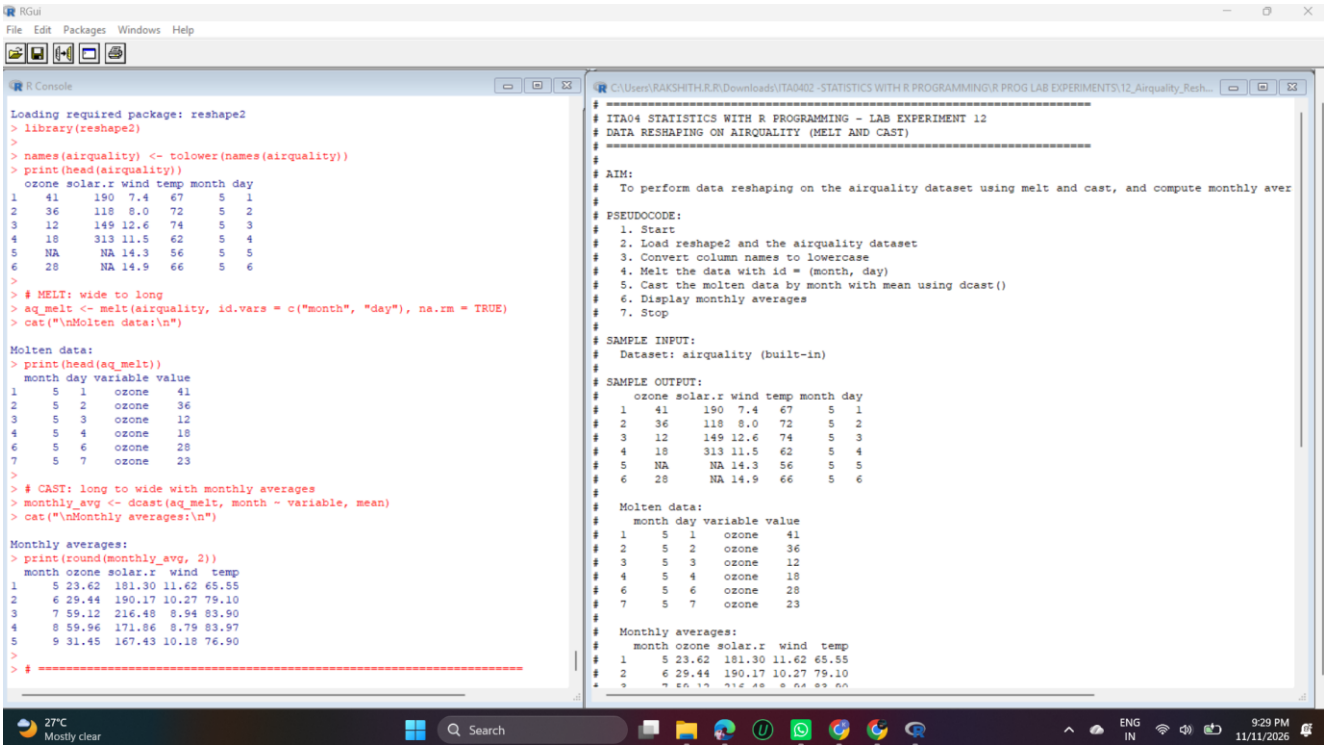
```
  ozone solar.r wind temp month day
1    41     190  7.4   67     5   1
2    36     118  8.0   72     5   2
3    12     149 12.6   74     5   3
4    18     313 11.5   62     5   4
5    NA      NA 14.3   56     5   5
6    28      NA 14.9   66     5   6


Molten data:
  month day variable value
1     5   1    ozone    41
2     5   2    ozone    36
3     5   3    ozone    12
```

```
4     5   4    ozone    18
6     5   6    ozone    28
7     5   7    ozone    23


Monthly averages:
  month ozone solar.r  wind  temp
1     5 23.62  181.30 11.62 65.55
2     6 29.44  190.17 10.27 79.10
3     7 59.12  216.48  8.94 83.90
4     8 59.96  171.86  8.79 83.97
5     9 31.45  167.43 10.18 76.90
```

Output Screenshot



Output – Experiment 12

Explanation

The column names are changed to lowercase with tolower(). melt() from the reshape2 package converts the data from wide to long format: month and day stay as identifiers, and ozone, solar.r, wind and temp become variable/value pairs. na.rm = TRUE removes missing values, which is why row 5 of the original data is missing in the molten output.

dcast(aq_melt, month ~ variable, mean) casts the long data back to wide format with one row per month and the mean of each variable.

The monthly averages show that ozone and temperature are highest in July and August (ozone about 59, temperature about 84) and lowest in May.

Experiment 13: Combine Arrays Row-wise

Question

13. Combine multiple arrays row-wise.

Code

```
arr1 <- array(1:6,   dim = c(2, 3))
arr2 <- array(7:12,  dim = c(2, 3))
arr3 <- array(13:18, dim = c(2, 3))

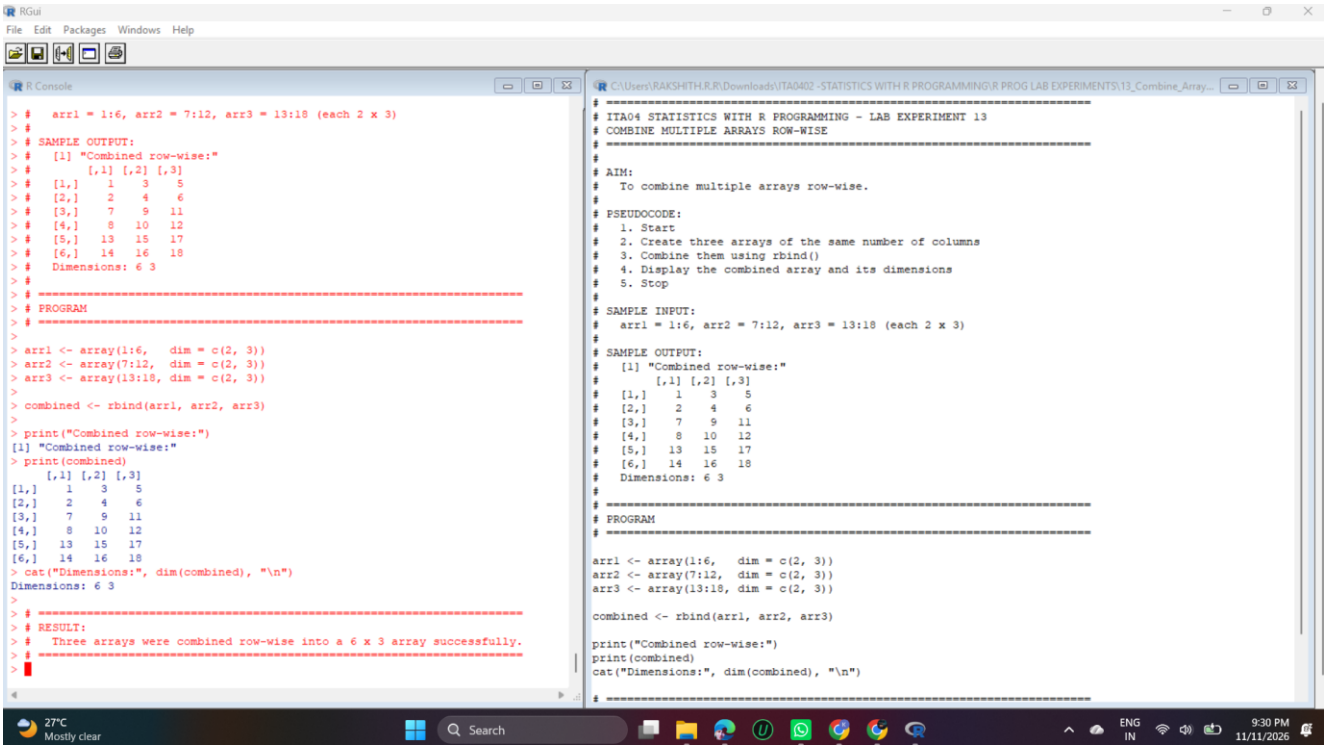

combined <- rbind(arr1, arr2, arr3)


print("Combined row-wise:")
print(combined)
cat("Dimensions:", dim(combined), "\n")
```

Output

```
[1] "Combined row-wise:"
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
[3,]    7    9   11
[4,]    8   10   12
[5,]   13   15   17
[6,]   14   16   18
Dimensions: 6 3
```

Output Screenshot



Output – Experiment 13

Explanation

array(..., dim = c(2, 3)) creates three arrays of 2 rows and 3 columns.

rbind() joins them one below the other (row-wise), giving 2 + 2 + 2 = 6 rows and 3 columns. Row-wise combining requires the same number of columns in every array.

dim() confirms the combined result is 6 × 3.

Experiment 14: ChickWeight Dataset

Question

14. Explore and manipulate ChickWeight dataset (sorting, melting, casting by Diet).

Code

```
library(reshape2)


cw <- as.data.frame(ChickWeight)
print(head(cw))
str(cw)


# Sorting (heaviest first)
cat("\nTop 5 heaviest records:\n")
print(head(cw[order(-cw$weight), ], 5))


# Melting
cw_melt <- melt(cw, id.vars = c("Chick", "Diet", "Time"), measure.vars = "weight")
cat("\nMolten data:\n")
print(head(cw_melt))


# Casting by Diet
cat("\nAverage weight by Diet:\n")
print(dcast(cw_melt, Diet ~ variable, mean))


# Casting by Diet and Time
cat("\nAverage weight by Diet at Time 0, 10, 20:\n")
sub <- cw_melt[cw_melt$Time %in% c(0, 10, 20), ]
tbl <- dcast(sub, Diet ~ Time, mean)
tbl[, -1] <- round(tbl[, -1], 2)
print(tbl)
```

Output

```
  weight Time Chick Diet
1     42    0     1    1
2     51    2     1    1
3     59    4     1    1
```

```
4    64    6    1    1
5    76    8    1    1
6    93   10    1    1
'data.frame':    578 obs. of  4 variables:
 $ weight: num  42 51 59 64 76 93 106 125 149 171 ...
 $ Time  : num  0 2 4 6 8 10 12 14 16 18 ...
 $ Chick : Ord.factor w/ 50 levels "18"<"16"<"15"<..: 15 15 15 15 15 15 15 15 15 15 ...
 $ Diet  : Factor w/ 4 levels "1","2","3","4": 1 1 1 1 1 1 1 1 1 1 ...
 - attr(*, "formula")=Class 'formula'  language weight ~ Time | Chick
  .. ..- attr(*, ".Environment")=<environment: R_EmptyEnv>
 - attr(*, "outer")=Class 'formula'  language ~Diet
  .. ..- attr(*, ".Environment")=<environment: R_EmptyEnv>
 - attr(*, "labels")=List of 2
  ..$ x: chr "Time"
  ..$ y: chr "Body weight"
 - attr(*, "units")=List of 2
  ..$ x: chr "(days)"
  ..$ y: chr "(gm)"

Top 5 heaviest records:
    weight Time Chick Diet
400    373   21    35    3
399    361   20    35    3
388    341   21    34    3
398    332   18    35    3
232    331   21    21    2

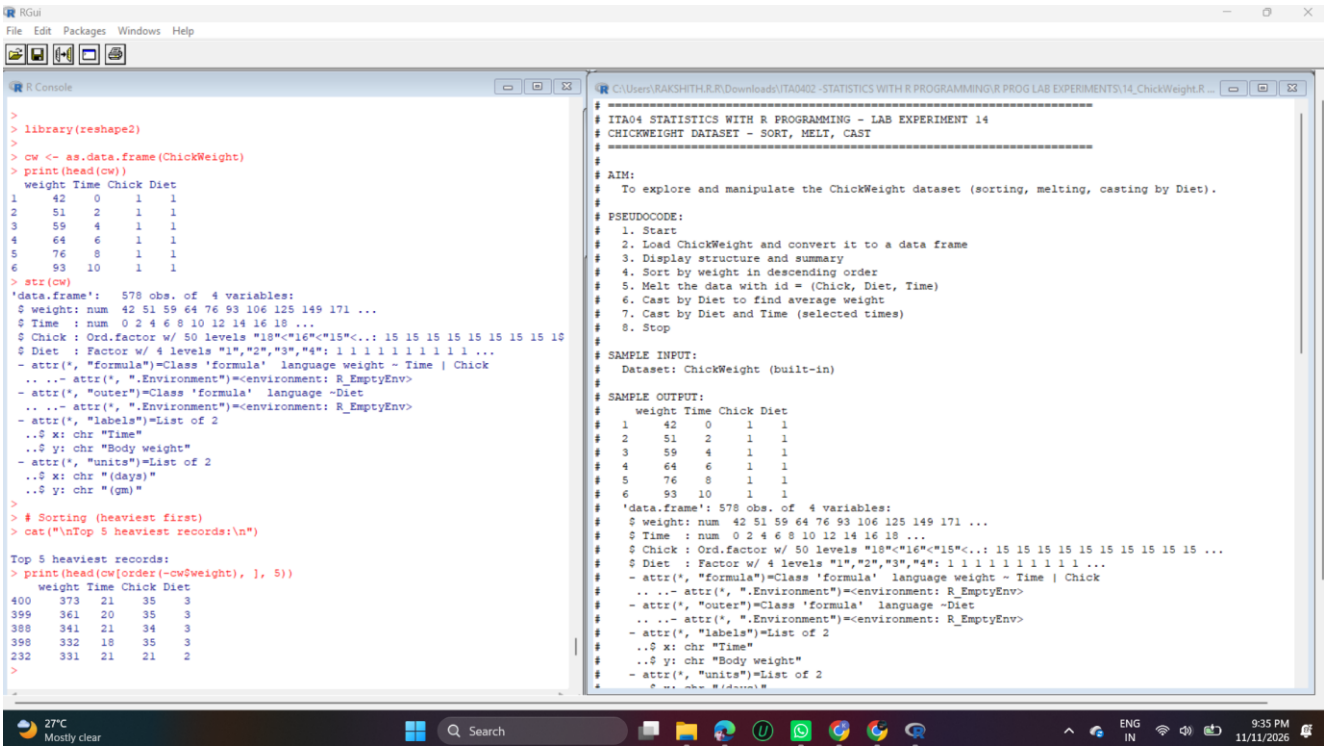
Molten data:
  Chick Diet Time variable value
1     1    1    0   weight    42
2     1    1    2   weight    51
3     1    1    4   weight    59
4     1    1    6   weight    64
5     1    1    8   weight    76
6     1    1   10   weight    93

Average weight by Diet:
  Diet   weight
1    1 102.6455
```

```
2    2 122.6167
3    3 142.9500
4    4 135.2627


Average weight by Diet at Time 0, 10, 20:
  Diet    0     10     20
1    1 41.4  93.05 170.41
2    2 40.7 108.50 205.60
3    3 40.8 117.10 258.90
4    4 41.0 126.00 233.89
```

Output Screenshot



Output – Experiment 14

Explanation

ChickWeight is a built-in grouped dataset (578 observations of weight, Time, Chick and Diet), converted to a plain data frame with as.data.frame().

Sorting: order(-cw$weight) lists the heaviest records first. The heaviest chick was Chick 35 on Diet 3 with 373 g at day 21.

Melting: melt() with id.vars = c("Chick", "Diet", "Time") and measure.vars = "weight" gives the long format with variable and value columns.

Casting: dcast(Diet ~ variable, mean) gives the average weight per diet. Diet 3 is the highest (142.95) and Diet 1 the lowest (102.65). The Diet × Time table shows that all diets start at about 41 g, and by day 20 Diet 3 is heaviest (258.9 g).

Experiment 15: EDA on iris

Question

15. Perform EDA on iris dataset: dimensions, summary, standard deviation, quantiles, grouping by Species, pivot table, categorical grouping with Sepal.Length categories.

Code

```
data(iris)
cat("Dimensions:", dim(iris), "\n\n")
print(head(iris))

cat("\nSummary:\n")
print(summary(iris))

cat("\nStandard deviation:\n")
print(sapply(iris[, 1:4], sd))

cat("\nQuantiles of Sepal.Length:\n")
print(quantile(iris$Sepal.Length))

cat("\nMean of each variable grouped by Species:\n")
print(aggregate(. ~ Species, data = iris, FUN = mean))

cat("\nPivot table (mean Petal.Length by Species):\n")
print(tapply(iris$Petal.Length, iris$Species, mean))

cat("\nSepal.Length categories:\n")
iris$SL_cat <- cut(iris$Sepal.Length, breaks = c(0, 5.5, 6.5, 10),
                   labels = c("Short", "Medium", "Long"))
print(table(iris$Species, iris$SL_cat))
```

Output

```
Dimensions: 150 5


  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
```

```
2          4.9         3.0          1.4          0.2  setosa
3          4.7         3.2          1.3          0.2  setosa
4          4.6         3.1          1.5          0.2  setosa
5          5.0         3.6          1.4          0.2  setosa
6          5.4         3.9          1.7          0.4  setosa


Summary:
 Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
 Median :5.800   Median :3.000   Median :4.350   Median :1.300
 Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
       Species
 setosa    :50
 versicolor:50
 virginica :50




Standard deviation:
Sepal.Length  Sepal.Width Petal.Length  Petal.Width
   0.8280661    0.4358663    1.7652982    0.7622377


Quantiles of Sepal.Length:
  0%  25%  50%  75% 100%
 4.3  5.1  5.8  6.4  7.9

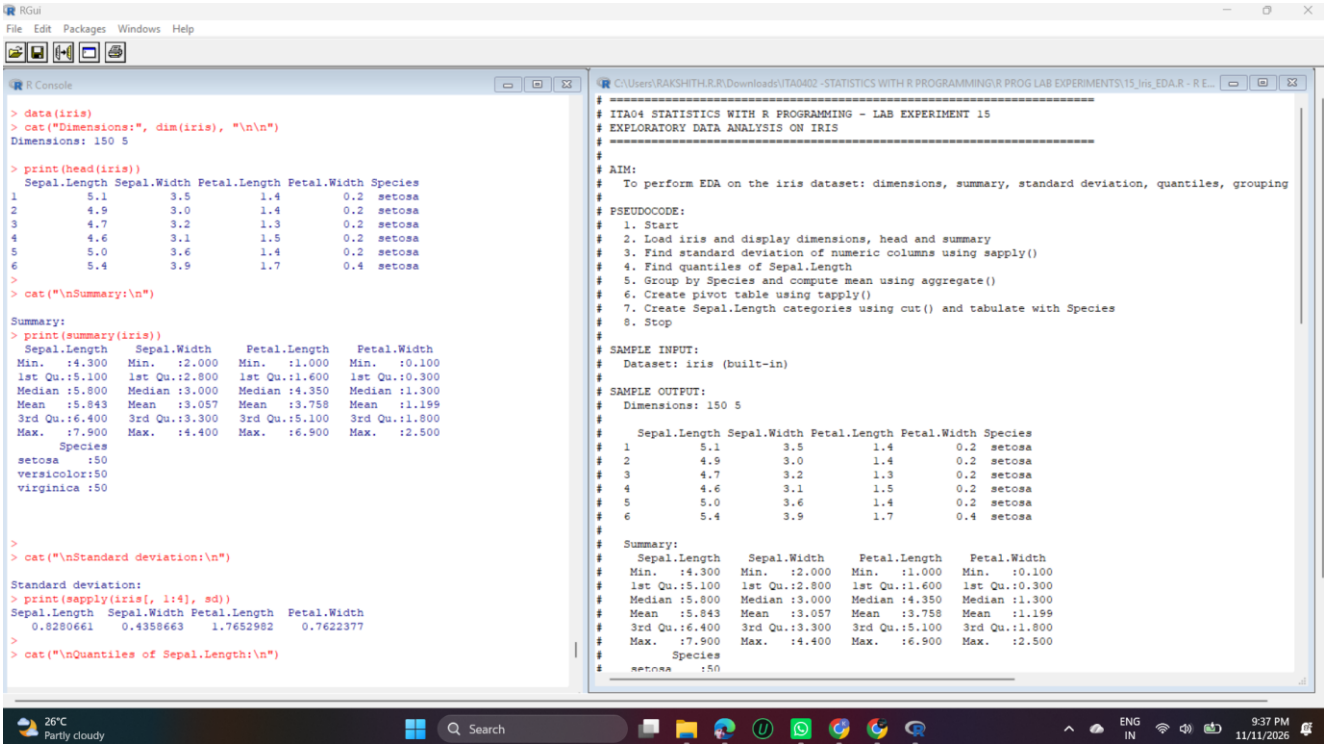

Mean of each variable grouped by Species:
     Species Sepal.Length Sepal.Width Petal.Length Petal.Width
1     setosa        5.006       3.428        1.462       0.246
2 versicolor        5.936       2.770        4.260       1.326
3  virginica        6.588       2.974        5.552       2.026
```

```
Pivot table (mean Petal.Length by Species):
   setosa versicolor  virginica
    1.462      4.260      5.552


Sepal.Length categories:


           Short Medium Long
setosa        47      3    0
versicolor    11     31    8
virginica      1     27   22
```

Output Screenshot



Output – Experiment 15

Explanation

dim() shows the dataset has 150 rows and 5 columns, head() shows the first six rows and summary() gives the min, quartiles, mean and max of each column plus the count per species.

sapply(iris[, 1:4], sd) gives the standard deviation of each numeric column. Petal.Length has the largest spread (1.77), so it varies the most. quantile() shows the 0%, 25%, 50%, 75% and 100% points of Sepal.Length.

aggregate(. ~ Species, FUN = mean) gives the mean of each variable per species (setosa has the smallest petals), and tapply() gives the same kind of pivot table for Petal.Length.

cut() converts Sepal.Length into the categories Short (up to 5.5), Medium (5.5–6.5) and Long (above 6.5). The cross-table shows that setosa is mostly Short, versicolor mostly Medium and virginica Medium or Long.

Experiment 16: USArrests Dataset

Question

16. Explore USArrests dataset: summary statistics, state with largest rape arrests, max & min murder rates, correlation among features, states above median assault arrests and bottom 25% for murder, visualization with histogram, density, scatterplots, bar graphs.

Code

```
data(USArrests)
print(head(USArrests))
cat("\nSummary:\n"); print(summary(USArrests))


cat("\nState with largest Rape arrests:", rownames(USArrests)[which.max(USArrests$Rape)], "\n")
cat("State with maximum Murder rate :", rownames(USArrests)[which.max(USArrests$Murder)],
   "(", max(USArrests$Murder), ")\n")
cat("State with minimum Murder rate :", rownames(USArrests)[which.min(USArrests$Murder)],
   "(", min(USArrests$Murder), ")\n")


cat("\nCorrelation among features:\n")
print(round(cor(USArrests), 2))


cat("\nStates with Assault above median:\n")
print(rownames(USArrests)[USArrests$Assault > median(USArrests$Assault)])


cat("\nStates in bottom 25% for Murder:\n")
print(rownames(USArrests)[USArrests$Murder <= quantile(USArrests$Murder, 0.25)])


# Visualization
par(mfrow = c(2, 2))
hist(USArrests$Murder, main = "Histogram of Murder", xlab = "Murder", col = "skyblue")
plot(density(USArrests$Assault), main = "Density of Assault", col = "red")
plot(USArrests$Assault, USArrests$Murder, main = "Assault vs Murder",
    xlab = "Assault", ylab = "Murder", pch = 19, col = "blue")
barplot(head(USArrests$Murder, 10), names.arg = head(rownames(USArrests), 10),
       las = 2, main = "Murder rate (first 10 states)", col = "orange")
par(mfrow = c(1, 1))
```

Output

```
          Murder Assault UrbanPop Rape
Alabama     13.2     236       58 21.2
Alaska      10.0     263       48 44.5
Arizona      8.1     294       80 31.0
Arkansas     8.8     190       50 19.5
California   9.0     276       91 40.6
Colorado     7.9     204       78 38.7


Summary:
     Murder          Assault         UrbanPop          Rape
 Min.   : 0.800   Min.   : 45.0   Min.   :32.00   Min.   : 7.30
 1st Qu.: 4.075   1st Qu.:109.0   1st Qu.:54.50   1st Qu.:15.07
 Median : 7.250   Median :159.0   Median :66.00   Median :20.10
 Mean   : 7.788   Mean   :170.8   Mean   :65.54   Mean   :21.23
 3rd Qu.:11.250   3rd Qu.:249.0   3rd Qu.:77.75   3rd Qu.:26.18
 Max.   :17.400   Max.   :337.0   Max.   :91.00   Max.   :46.00


State with largest Rape arrests: Nevada
State with maximum Murder rate : Georgia ( 17.4 )
State with minimum Murder rate : North Dakota ( 0.8 )


Correlation among features:
          Murder Assault UrbanPop Rape
Murder      1.00    0.80     0.07 0.56
Assault     0.80    1.00     0.26 0.67
UrbanPop    0.07    0.26     1.00 0.41
Rape        0.56    0.67     0.41 1.00


States with Assault above median:
 [1] "Alabama"        "Alaska"         "Arizona"        "Arkansas"
 [5] "California"     "Colorado"       "Delaware"       "Florida"
 [9] "Georgia"        "Illinois"       "Louisiana"      "Maryland"
[13] "Michigan"       "Mississippi"    "Missouri"       "Nevada"
[17] "New Mexico"     "New York"       "North Carolina" "Rhode Island"
[21] "South Carolina" "Tennessee"      "Texas"          "Wyoming"
```

States in bottom 25% for Murder:

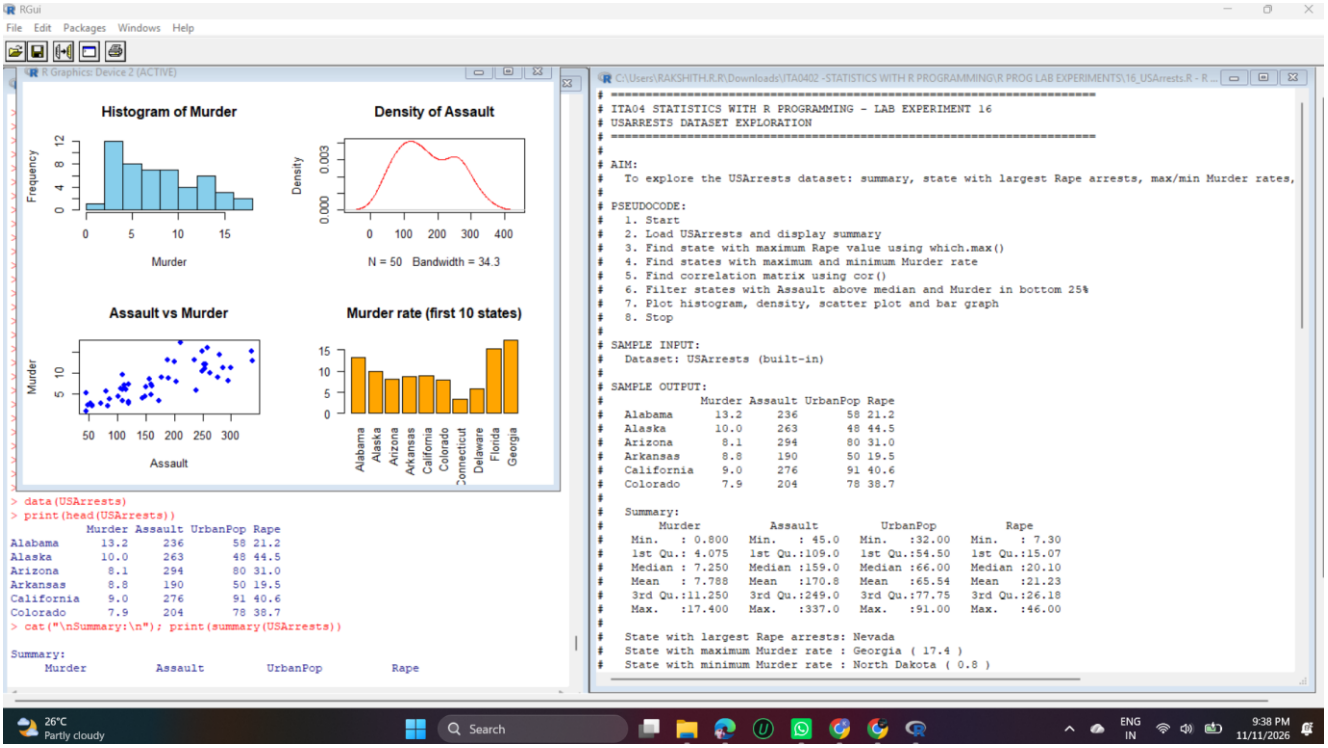
[1] "Connecticut" "Idaho" "Iowa" "Maine"

[5] "Minnesota" "New Hampshire" "North Dakota" "Rhode Island"

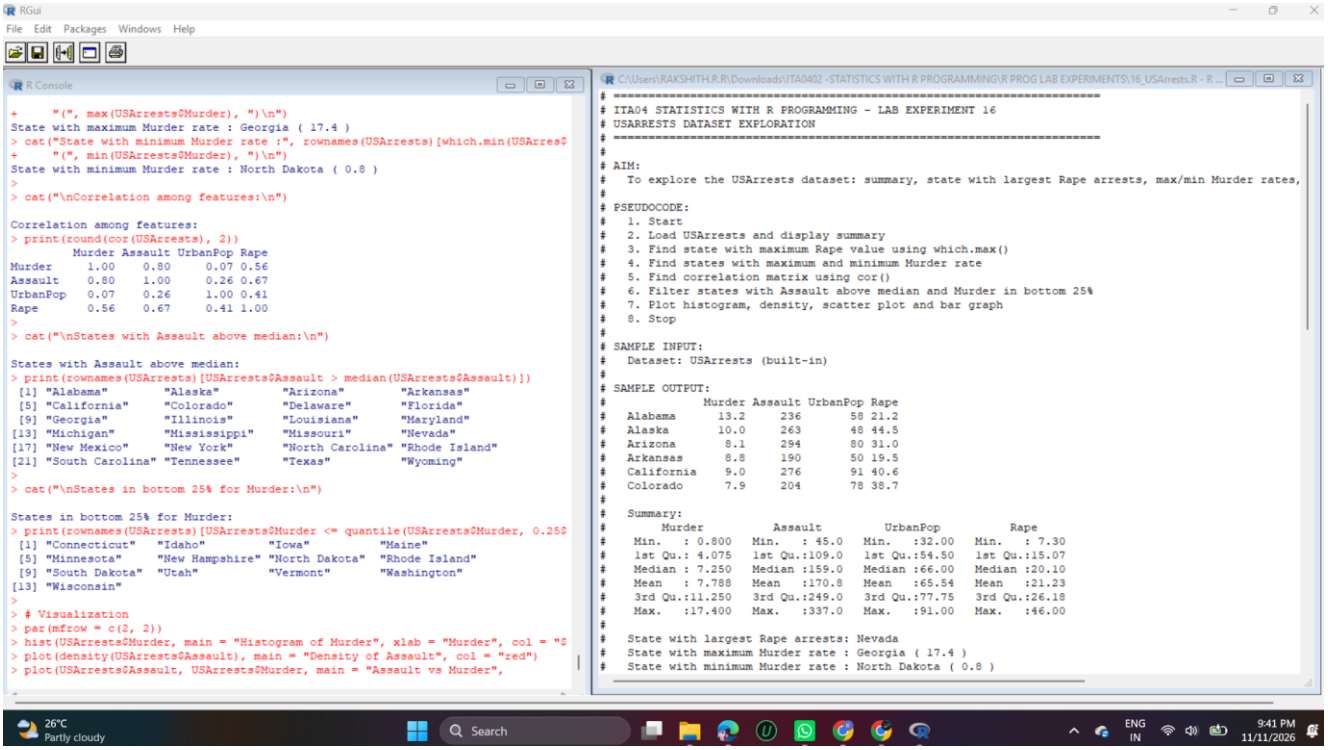
[9] "South Dakota" "Utah" "Vermont" "Washington"

[13] "Wisconsin"

Output Screenshots



Output 1 of 2 – Experiment 16



Output 2 of 2 – Experiment 16

Explanation

summary() gives the basic statistics. which.max() and which.min() return the position of the largest and smallest value, and rownames() converts that position into the state name: Nevada has the largest Rape arrests (46), Georgia the maximum Murder rate (17.4) and North Dakota the minimum (0.8).

cor() gives the correlation matrix. Murder and Assault are strongly related (0.80), while Murder and UrbanPop are almost unrelated (0.07).

Logical indexing filters the states: 24 states have an Assault value above the median (159), and 13 states fall in the bottom 25% for Murder (at or below 4.075).

The four plots show the distribution of Murder (histogram), the shape of Assault (density curve with two peaks), the positive relationship between Assault and Murder (scatter plot) and the murder rate of the first 10 states (bar graph).

Experiment 17: Titanic Dataset

Question

17. Explore Titanic dataset: bar chart of survival vs class, modify plot by gender, histogram of Age.

Code

```
if (file.exists("titanic.csv")) {
  titanic <- read.csv("titanic.csv")
} else {
  titanic <- read.csv("https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv")
}
cat("Rows:", nrow(titanic), " Columns:", ncol(titanic), "\n")


# Bar chart: Survival vs Class
t1 <- table(Survived = titanic$Survived, Class = titanic$Pclass)
print(t1)
barplot(t1, beside = TRUE, col = c("tomato", "seagreen"),
        main = "Survival by Passenger Class", xlab = "Class", ylab = "Passengers",
        legend.text = c("Died", "Survived"))


# Modified plot: by gender
t2 <- table(Survived = titanic$Survived, Sex = titanic$Sex)
print(t2)
barplot(t2, beside = TRUE, col = c("tomato", "seagreen"),
        main = "Survival by Gender", xlab = "Gender", ylab = "Passengers",
        legend.text = c("Died", "Survived"))

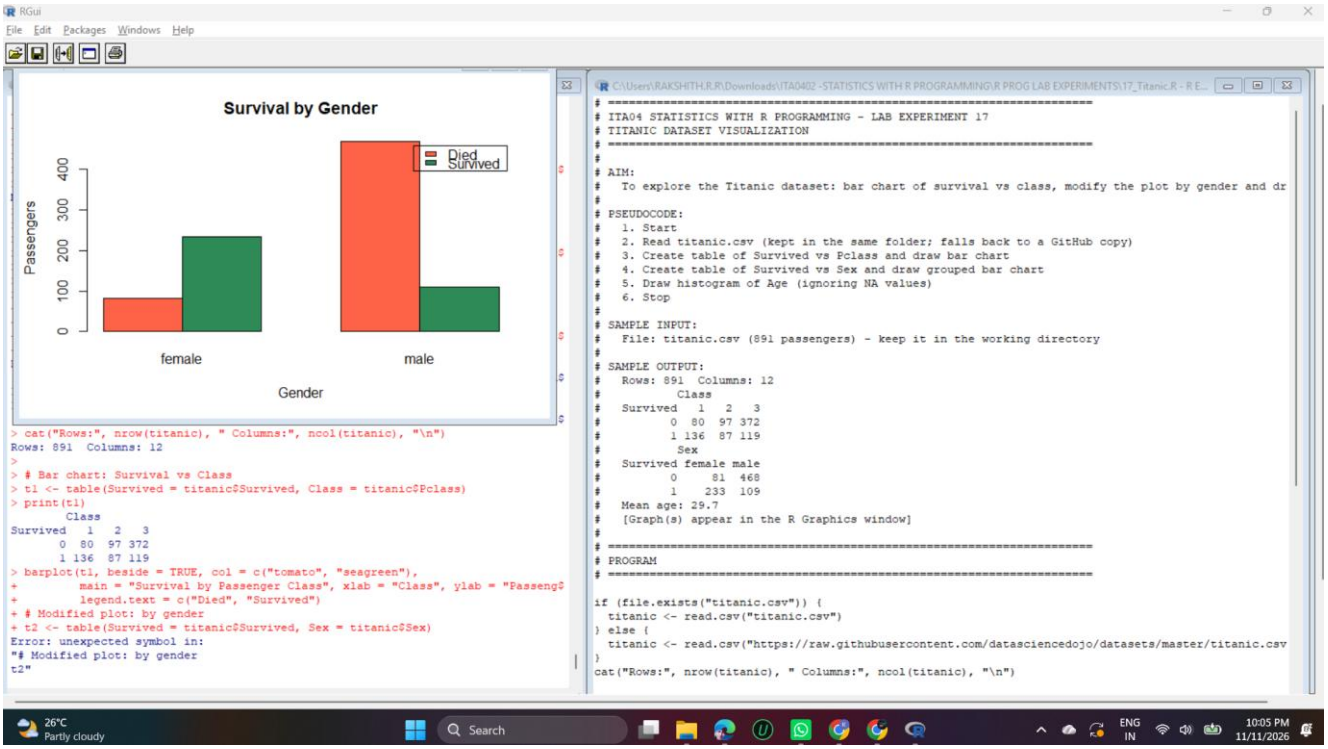

# Histogram of Age
hist(titanic$Age, breaks = 15, col = "skyblue", main = "Histogram of Age", xlab = "Age")
cat("Mean age:", round(mean(titanic$Age, na.rm = TRUE), 2), "\n")
```

Output

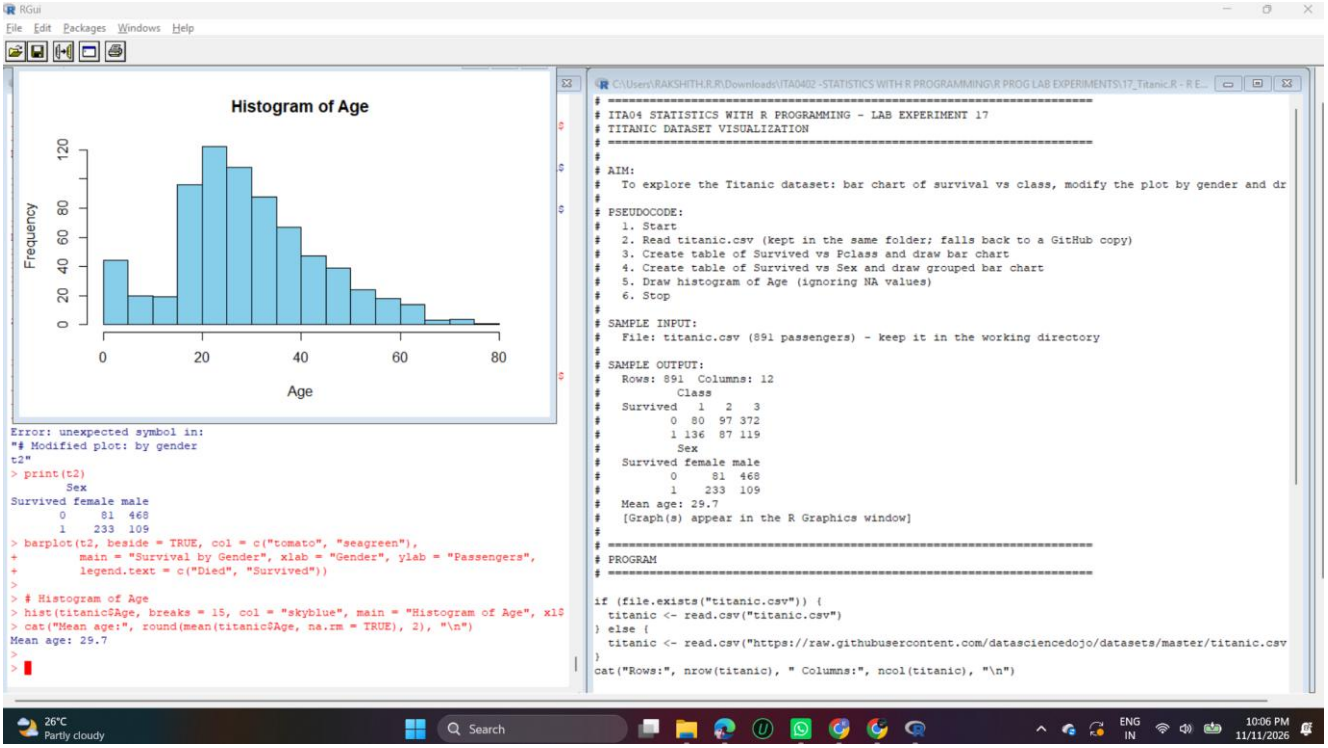
```
Rows: 891  Columns: 12
        Class
Survived   1   2   3
       0  80  97 372
       1 136  87 119
        Sex
```

```
Survived female male
    0     81  468
    1    233  109
Mean age: 29.7
```

Output Screenshots



Output 1 of 2 – Experiment 17



Output 2 of 2 – Experiment 17

Explanation

The program reads titanic.csv (891 passengers, 12 columns) and uses table() to count survival against class and against gender.

Survival by class: first class had 136 survivors against 80 deaths, while third class had 119 survivors against 372 deaths. Survival chances fell with class.

Survival by gender: 233 females survived and 81 died, whereas only 109 males survived and 468 died. barplot(..., beside = TRUE) draws the Died and Survived bars side by side with a legend.

The histogram of Age (missing ages ignored) shows most passengers were between 20 and 40 years old, and the mean age is 29.7 years.

Experiment 18: Basic Graphs

Question

18. Create graphs in R: boxplot, histogram, bar plot, line chart, scatter plot.

Code

```
par(mfrow = c(2, 3))


boxplot(mpg ~ cyl, data = mtcars, main = "Boxplot", xlab = "Cylinders",
        ylab = "Mileage", col = "lightblue")


hist(mtcars$mpg, main = "Histogram", xlab = "Mileage", col = "orange")


counts <- table(mtcars$gear)
barplot(counts, main = "Bar Plot", xlab = "Gears", ylab = "Cars", col = "green")


sales <- c(20, 35, 30, 45, 60, 55)
plot(1:6, sales, type = "o", main = "Line Chart", xlab = "Month", ylab = "Sales", col = "red")

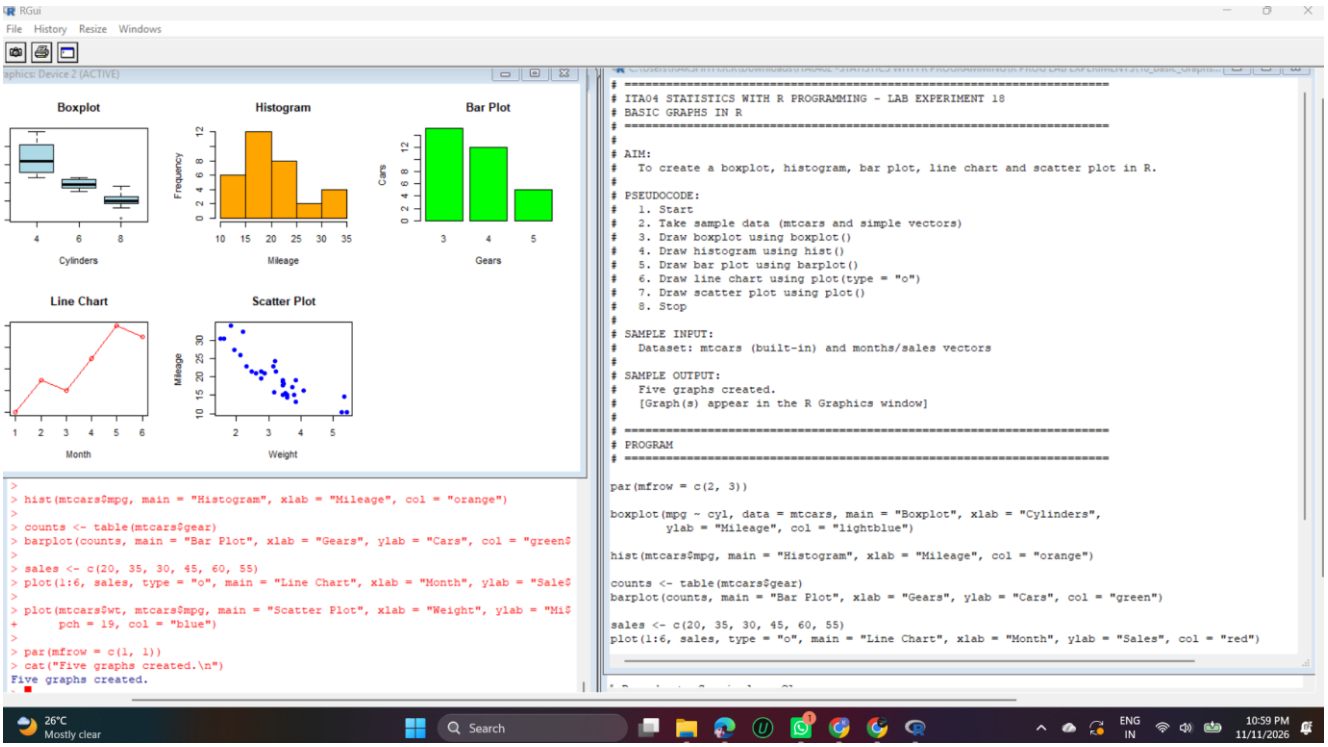

plot(mtcars$wt, mtcars$mpg, main = "Scatter Plot", xlab = "Weight", ylab = "Mileage",
     pch = 19, col = "blue")


par(mfrow = c(1, 1))
cat("Five graphs created.\n")
```

Output

```
Five graphs created.
```

Output Screenshot



Output – Experiment 18

Explanation

par(mfrow = c(2, 3)) creates a grid of 2 rows and 3 columns so that five graphs can be shown in one window.

The boxplot compares mileage for 4, 6 and 8 cylinders (mileage falls as cylinders increase). The histogram shows the distribution of mileage, with most cars between 15 and 25 mpg. The bar plot counts cars by number of gears (3 gears is most common).

The line chart plots sales over six months with plot(type = "o"), and the scatter plot shows that heavier cars have lower mileage (a negative relationship).

Experiment 19: Simple Linear Regression

Question

19. Build a regression model on advertising dataset (Sales ~ Spend) and predict Sales.

Code

```
advertising <- data.frame(
  Spend = c(10, 20, 30, 40, 50, 60, 70, 80, 90, 100),
  Sales = c(25, 32, 41, 48, 57, 66, 71, 82, 90, 98)
)

model <- lm(Sales ~ Spend, data = advertising)
print(summary(model))

cat("\nCoefficients:\n")
print(coef(model))

new_data <- data.frame(Spend = 120)
pred <- predict(model, new_data)
cat("\nPredicted Sales for Spend = 120 :", round(pred, 2), "\n")

plot(advertising$Spend, advertising$Sales, pch = 19, col = "blue",
     main = "Sales vs Spend", xlab = "Spend", ylab = "Sales")
abline(model, col = "red", lwd = 2)
```

Output

```
Call:
lm(formula = Sales ~ Spend, data = advertising)

Residuals:
   Min     1Q  Median     3Q    Max
-2.2364 -0.3167  0.3424  0.5667  0.9212

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept) 16.13333    0.68106   23.69 1.07e-08 ***
Spend        0.81576    0.01098   74.32 1.20e-12 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

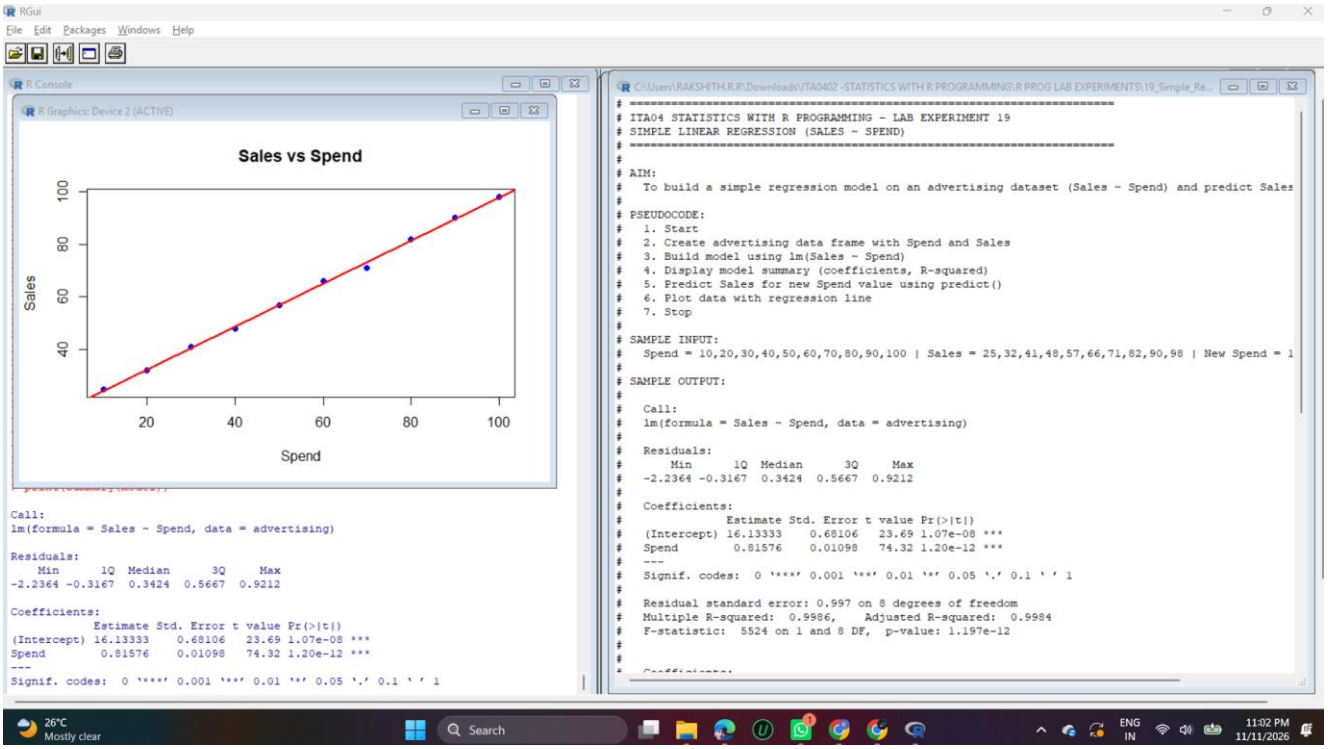

Residual standard error: 0.997 on 8 degrees of freedom
Multiple R-squared:  0.9986,    Adjusted R-squared:  0.9984
F-statistic:  5524 on 1 and 8 DF,  p-value: 1.197e-12



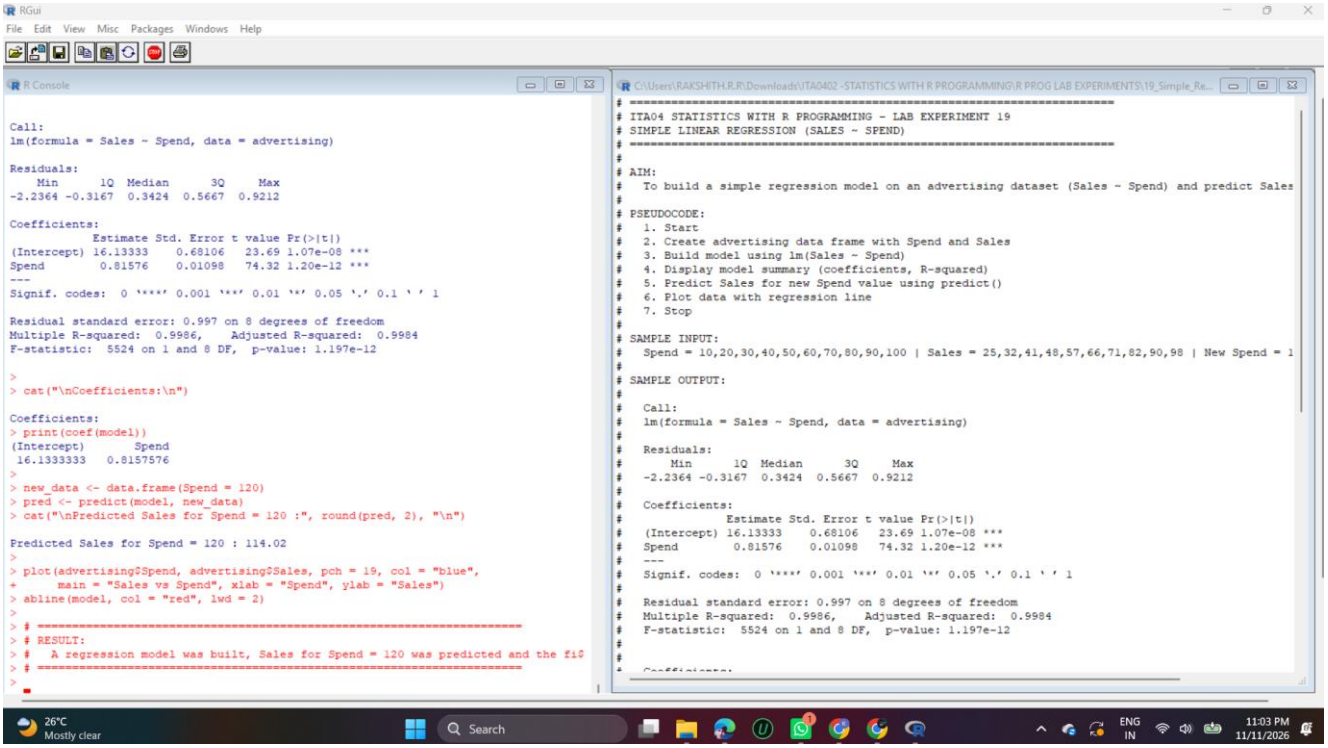
Coefficients:
(Intercept)       Spend
 16.1333333   0.8157576


Predicted Sales for Spend = 120 : 114.02
```

Output Screenshots



Output 1 of 2 – Experiment 19



Output 2 of 2 – Experiment 19

Explanation

lm(Sales ~ Spend) fits a simple linear regression of Sales on Spend. The fitted equation is Sales = 16.13 + 0.8158 × Spend, so each unit increase in Spend raises Sales by about 0.82.

Multiple R-squared is 0.9986, so the model explains 99.86% of the variation in Sales, and the p-value (1.2e-12) shows that Spend is highly significant.

predict() applies the model to a new Spend of 120 and returns Sales = 16.13 + 0.8158 × 120 = 114.02. The scatter plot with abline() shows the points lying very close to the red regression line.

Experiment 20: Multiple Regression

Question

20. Create multiple regression model using ChickWeight dataset with “Time” and “Diet” as predictors; predict weight and compute model error.

Code

```
cw <- as.data.frame(ChickWeight)


model <- lm(weight ~ Time + Diet, data = cw)
print(summary(model))


# Prediction for a new chick
new_chick <- data.frame(Time = 10, Diet = factor("1", levels = levels(cw$Diet)))
cat("\nPredicted weight (Time=10, Diet=1):", round(predict(model, new_chick), 2), "\n")


# Model error
pred <- predict(model, cw)
error <- cw$weight - pred
rmse <- sqrt(mean(error^2))
mae  <- mean(abs(error))


cat("\nRMSE :", round(rmse, 2), "\n")
cat("MAE  :", round(mae, 2), "\n")
cat("R-squared:", round(summary(model)$r.squared, 4), "\n")
```

Output

```

Call:
lm(formula = weight ~ Time + Diet, data = cw)


Residuals:
    Min      1Q  Median      3Q     Max
-136.851  -17.151   -2.595   15.033  141.816


Coefficients:
```

```
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  10.9244     3.3607   3.251  0.00122 **
Time          8.7505     0.2218  39.451  < 2e-16 ***
Diet2        16.1661     4.0858   3.957 8.56e-05 ***
Diet3        36.4994     4.0858   8.933  < 2e-16 ***
Diet4        30.2335     4.1075   7.361 6.39e-13 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

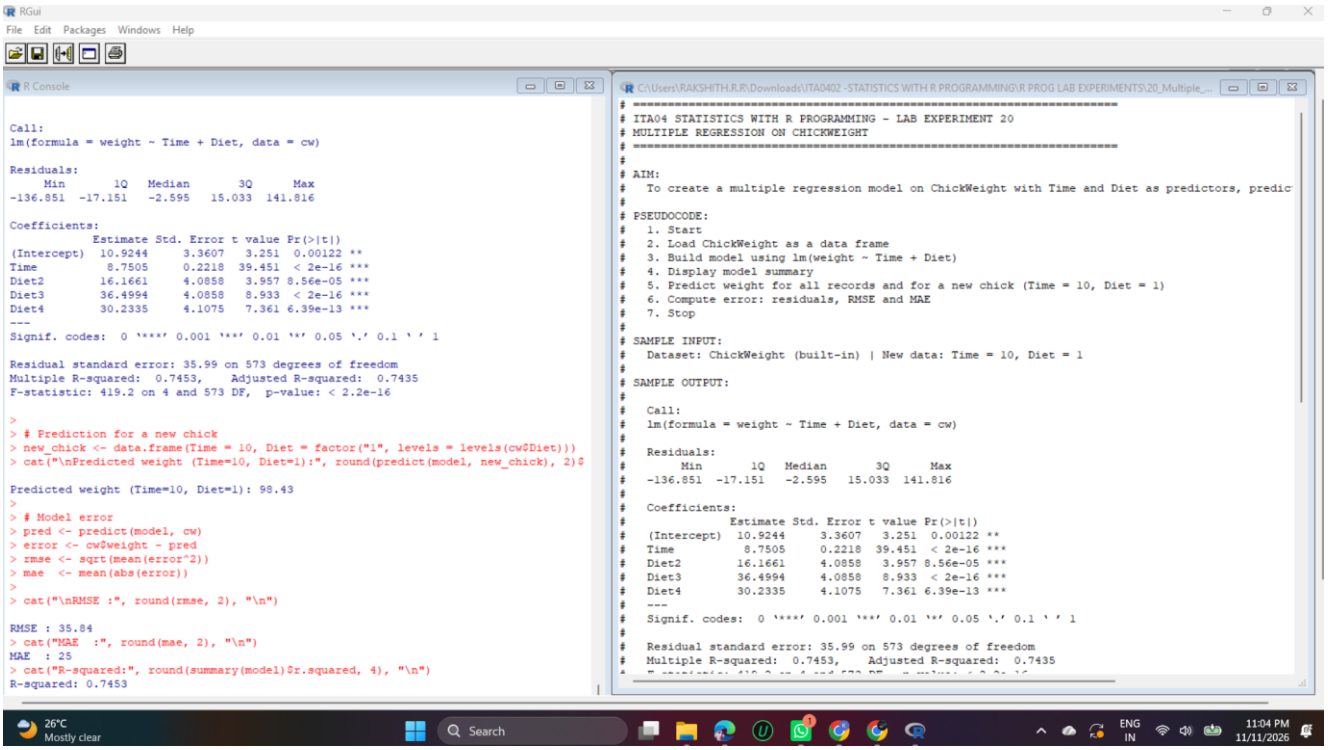

Residual standard error: 35.99 on 573 degrees of freedom
Multiple R-squared:  0.7453,    Adjusted R-squared:  0.7435
F-statistic: 419.2 on 4 and 573 DF,  p-value: < 2.2e-16



Predicted weight (Time=10, Diet=1): 98.43


RMSE : 35.84
MAE  : 25
R-squared: 0.7453
```

Output Screenshot



Output – Experiment 20

Explanation

lm(weight ~ Time + Diet) uses two predictors. Diet is a factor, so R creates the dummy variables Diet2, Diet3 and Diet4 and treats Diet 1 as the baseline.

The coefficient of Time (8.75) means a chick gains about 8.75 g per day. Compared with Diet 1, Diet 2 adds 16.2 g, Diet 3 adds 36.5 g (the best diet) and Diet 4 adds 30.2 g. Multiple R-squared is 0.7453, so the model explains about 74.5% of the variation.

The prediction for Time = 10 and Diet = 1 is $10.92 + 8.75 \times 10 = 98.43$ g.

Model error: RMSE = 35.84 g and MAE = 25 g. On average the predictions differ from the actual weights by about 25 g, and RMSE is larger because it penalises large errors more.

Experiment 21: Logistic Regression on iris

Question

21. Randomly split iris dataset into train/test (80/20), build logistic regression (Species ~ Petal.Length + Petal.Width), predict, and evaluate with confusion matrix.

Code

```
# NOTE: glm(family = binomial) works only for 2 classes. Iris has 3 species,
# so multinomial logistic regression (nnet::multinom) is used.
library(nnet)


set.seed(123)
idx   <- sample(1:nrow(iris), 0.8 * nrow(iris))
train <- iris[idx, ]
test  <- iris[-idx, ]
cat("Training rows:", nrow(train), " Testing rows:", nrow(test), "\n")


model <- multinom(Species ~ Petal.Length + Petal.Width, data = train, trace = FALSE)
print(summary(model)$coefficients)


pred <- predict(model, test)


cm <- table(Actual = test$Species, Predicted = pred)
cat("\nConfusion Matrix:\n")
print(cm)


accuracy <- sum(diag(cm)) / sum(cm)
cat("\nAccuracy:", round(accuracy * 100, 2), "%\n")
```

Output

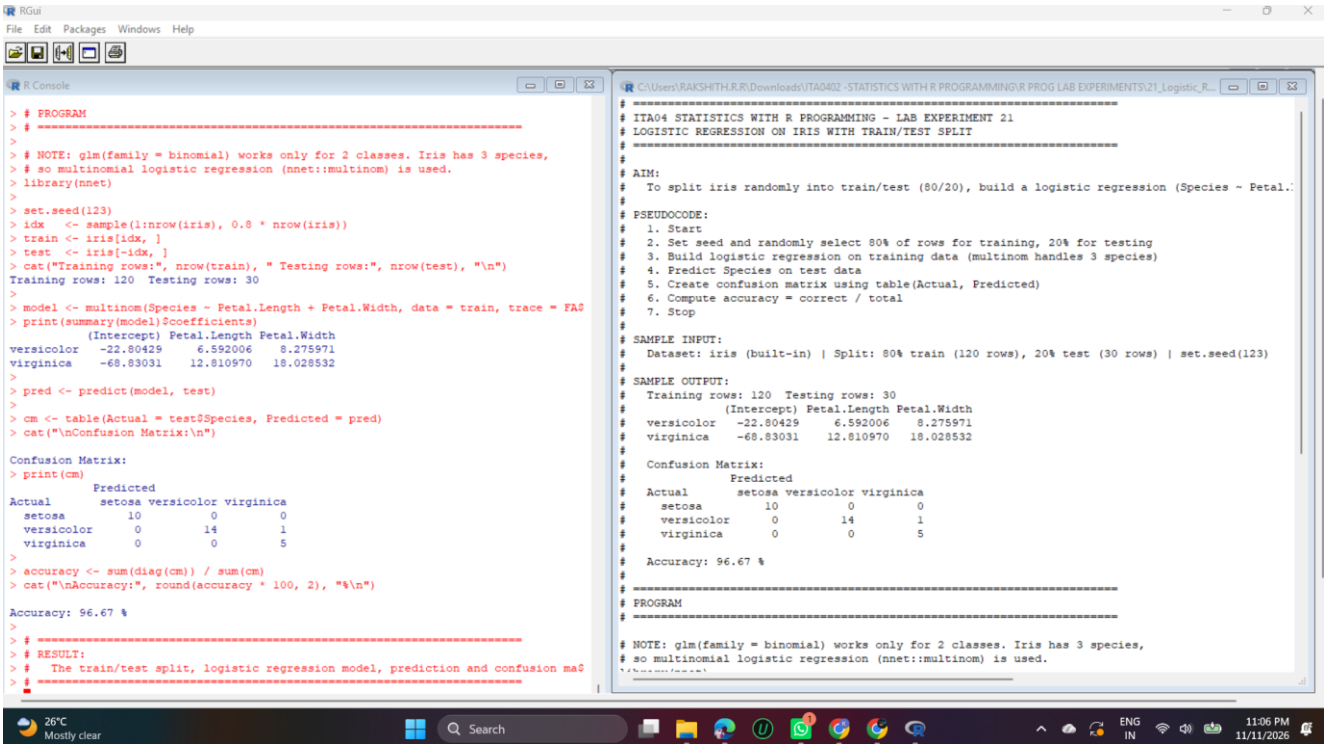
```
Training rows: 120  Testing rows: 30
           (Intercept) Petal.Length Petal.Width
versicolor   -22.80429     6.592006    8.275971
virginica    -68.83031    12.810970   18.028532


Confusion Matrix:
            Predicted
```

Actual	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	14	1
virginica	0	0	5

Accuracy: 96.67 %

Output Screenshot



Output – Experiment 21

Explanation

set.seed(123) makes the random split repeatable. sample() picks 80% of the row numbers (120 rows) for training and the remaining 30 rows are used for testing.

glm(family = binomial) works only for two classes, and iris has three species, so multinomial logistic regression (nnet::multinom) is used with Petal.Length and Petal.Width as predictors. The coefficients are given for versicolor and virginica relative to setosa.

predict() classifies the test data, and table(Actual, Predicted) forms the confusion matrix. The diagonal values (10, 14, 5) are correct predictions, and only one versicolor flower was wrongly predicted as virginica.

Accuracy = (10 + 14 + 5) / 30 = 96.67%, so the model separates the three species very well.